
The Elicita Handbook

Elicita helps you ask the question you would otherwise have missed — while the client is still in the room — and then writes up what was agreed, quoting the words that were actually said. This is a guide to using it. It goes in the order you will meet it: setting it up, preparing for an engagement, the meeting itself, and what comes out afterwards. Every screen is shown. It assumes nothing about how the software is built. Where something is not finished, it says so plainly rather than describing an intention in the same voice as a fact.

Contents

GETTING STARTED

Elicta Quick Start	3
What Elicta Does	17
Setting It Up	19
Getting It Onto People's Machines	26

BEFORE A MEETING

Preparing for an Engagement	28
Starting a Meeting	33

DURING THE MEETING

The Panel and the Suggestion	38
Controlling the Recording	42
Running a Meeting in Two Languages	46
When the Connection Drops	48

AFTER THE MEETING

Checking the Recording	50
Getting the Write-Up	54
Carrying It Into the Next Meeting	59

RUNNING IT WELL

Judging Whether the Suggestions Are Any Good	61
What It Will Not Do	64
What Works Today	65

GETTING STARTED

Elicita Quick Start

Your first hour with Elicita, in the order you will meet it.

Elicita helps you ask the question you would otherwise have missed — while the client is still in the room — and then writes up what was agreed, quoting the words that were actually said.

The one thing to understand before you start: **almost everything that makes Elicita useful happens before anyone sits down.** The thinking is done days in advance so that during the meeting it only has to pick the right question from work already finished. If you do nothing else from this guide, do steps 2 to 5.

Read this in one sitting; the whole first pass takes about an hour, most of which is waiting for the question bank to draft.

The rest of this book walks the same path slowly, one screen at a time. This chapter is the short way through it.

Before you begin

You need one thing that somebody has to give you: **a credential for an AI provider.** Elicita uses outside services for two jobs — turning speech into text, and doing the writing-up — and the question drafting is the one you will hit in your first hour.

Two things you do *not* need:

- **A database.** Everything an engagement remembers is written to a single file on your machine as you work. Nothing to install.
 - **A Microsoft 365 registration** — unless you want to attach documents by SharePoint or OneDrive *link*. Dropping the files in needs nothing set up at all, and is the way in that always works.
-

Step 1 — Point Elicita at the services it uses

Settings. Once, per machine. Five minutes.

On first run the Settings screen says plainly that nothing is configured, rather than looking ready and failing later. A tab whose service still needs something carries a small mark, so what is left to do can be read from the row of tabs before you open any of them.

SERVICE

Settings

These settings are held in memory and will be lost when the service restarts.

Claude • **Speech** • **Recording** • **Documents** • **Storage**

Claude Needs a credential

Drafts the question bank before the meeting and writes the debrief after it. Nothing during the meeting waits on it.

Route Claude calls through

Calls api.anthropic.com with the credential below.

Anthropic

Authenticate with

Whichever your organisation issues. Only the one selected here is sent, and only its field is shown.

API key **OAuth token**

Anthropic API key

A key issued from the Anthropic console (starts sk-ant-).

Not configured Test

No key is stored yet.

Model

Used for the context compiler and the debrief pipeline.

claude-opus-5

Save

Before anything is configured

The screen is a row of tabs with one thing behind each. Most are an outside service, and that service's credential sits behind its own tab under the name of the company that issued it — there is no list of keys to work out which goes where. The rest are choices about your own installation, and you can leave those as they are for now.

Fill in the first tab, which is the service that does the writing-up. You can point Elicita at the company that makes the service directly, at Amazon's, Google's or Microsoft's hosted versions, or at your organisation's own gateway. Choose Amazon or Google and no key is asked for at all — Elicita uses the permissions your cloud account already grants it, and the credential fields do not appear rather than sitting there greyed out.

The screenshot shows the 'Settings' screen for the 'Claude' service. At the top, there's a 'SERVICE' header with a logo. Below it, the 'Settings' title is prominent. A horizontal menu contains tabs for 'Claude', 'Speech', 'Recording', 'Documents', and 'Storage'. The 'Claude' tab is active, showing a 'Ready' status. A description states: 'Drafts the question bank before the meeting and writes the debrief after it. Nothing during the meeting waits on it.' The 'Route Claude calls through' section shows a dropdown menu set to 'Anthropic'. The 'Authenticate with' section has two buttons: 'API key' (selected) and 'OAuth token'. The 'Anthropic API key' section shows a status of 'in use', a description 'A key issued from the Anthropic console (starts sk-ant-).', a text field containing 'Configured — ends x7q2', and 'Test' and 'Clear' buttons. A note below says 'A key is stored. Leave this blank to keep it.' The 'Model' section has a text field containing 'claude-opus-5'. At the bottom right is a 'Save' button.

Credentials configured, with the one in use marked

Then press **Test**. It really tests — it checks the credential against the service rather than reporting "configured" as though that meant "working".

Four behaviours here will look odd until you know why:

What you see	Why
The key field is always empty	Elicita never sends a stored key back to the screen. It shows the last four characters instead, which is enough to tell one key from another.
Leaving it blank keeps the old one	So saving an unrelated setting cannot wipe your key by accident.
Clear is a separate button	Removing a credential is its own deliberate action, because it cannot be undone.

What you see	Why
Key or token, never both boxes	You say which kind you have and only that one is asked for.

Transcription services can wait. They are set up separately, and in this build nothing is connected to them yet — see *What to expect* below. Skip them on your first pass.

Step 2 — Add your client

Engagements. Two minutes.

Elicita opens on your clients. Each one is an **engagement**, and adding one asks three things: who they are, the sector, and the commercial shape of the work.

The screenshot shows a dark-themed interface titled 'Your clients' under the 'ENGAGEMENTS' header. It features a list of two existing engagements: 'Northwind Logistics' (Freight and logistics · Fixed-price discovery, three meetings) and 'Calder & Rowe' (Professional services · Matter-management replacement). Each entry has a 'Remove' button. Below the list, a paragraph explains that opening an engagement makes it the client for other screens, and removing it takes it off the list without erasing data. Under the 'NEW ENGAGEMENT' section, there is a form with three fields: 'Client organisation' (filled with 'Northwind Logistics'), 'Sector' (filled with 'Freight and logistics'), and 'Commercial context' (filled with 'Fixed-price discovery, three meetings'). A final paragraph states that these three fields are required to create an engagement. A 'Create engagement' button is at the bottom right.

Your clients, and the form for adding one

That is not filing. It is what lets Elicita work out which languages to expect in the room, so nobody has to be asked to choose one before a meeting starts.

Opening a client makes it the one every other screen is about.

The single most common confusion. A screen that looks empty is far more often pointed at a different client than genuinely empty. The switcher at the top of the window, beside the screen's name, follows you everywhere — that is the first place to check. Your choice is remembered between sessions.

Step 3 — Add its first meeting

One minute.

An engagement is a client. A **meeting** is one conversation with them — and everything after this point (recording, the live panel, the write-up) is about a meeting, so an engagement needs at least one before any of it applies.

Adding one asks a single question: how the audio will reach Elicita. A line-in from the meeting machine, a silent join that takes the call audio, or a microphone. You can change it later.

Step 4 — Give it what you already have

Preparation screen. Ten to twenty minutes, and the highest-value time you will spend.

Documents. Scoping decks, throughput studies, the last proposal. Drop the files straight onto the screen, or pick them with the button beside the drop area. Elicita reads what is *inside* them — Word, PowerPoint, Excel and PDF alike, not just the filename.

Tag each one, because the tag changes how Elicita treats it:

Tag	What it means
Ground truth	Established fact. If a client contradicts it, that is worth interrupting a meeting for.
Hypothesis	Our assumption, not theirs. Produces questions to verify, not contradictions to flag.
Superseded	Kept for background, never used to challenge anyone.

You can also attach a document by its SharePoint, OneDrive or Teams link — but that is the one thing here that must be set up first, and until it is, pasting a link is refused with a message saying so. That refusal is deliberate: a link recorded but never read would sit in the list looking like preparation that had happened.

The client's own words. Product names, internal systems, their jargon — the words a transcription service has no way of knowing.

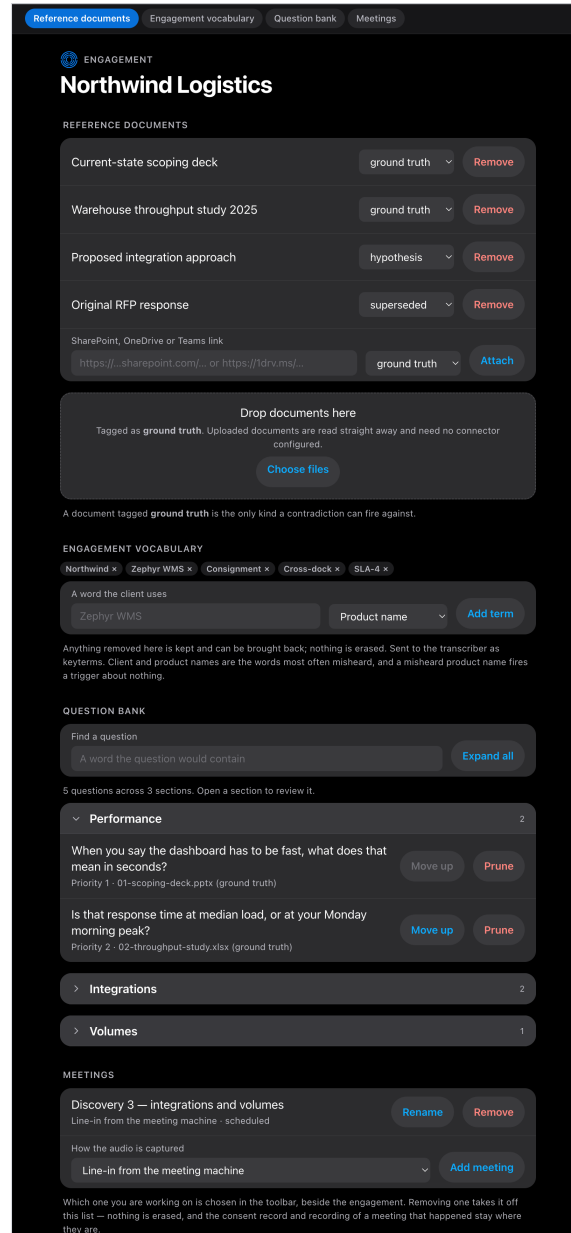
This is the most effective single thing you can do for accuracy, and it is worth more than any other preparation on this screen. A misheard product name does not just look sloppy: it reads to Elicita as something brand new, and it will interrupt you to ask about a system that does not exist. A typo in this list is worth correcting the moment you notice it — a wrong word handed to the transcription service is worse than no word at all, because it makes the transcript confidently wrong rather than merely uncertain.

Nothing you remove here is erased. A document's **Remove**, a word's small ✕, and removing the whole engagement all just stop it appearing; the record is kept. So tidy freely — but know that removing a client is *not* how you would honour a request to erase their data. Elicita does not do that yet.

Step 5 — Draft the questions, then read them

Press Compile, then go and make coffee.

Elicita reads everything you gave it and drafts the questions worth asking, grouped by the section of your requirements template each one serves.



The preparation screen: documents, vocabulary, and the question bank grouped by section

Drafting is slow work, so it is handed off as a job that finishes some minutes later; Elicita goes back for the result on its own and fills the bank when it arrives. **Compile before you make coffee, not while the client is waiting.** While it is working the screen says so, and will not let you start a second one on top of it.

Then do the part that matters: **read the questions, move the ones that matter up, and prune the ones that miss.** Pruning is not deletion — a pruned question is kept as a record of your judgement and stays pruned in every later meeting, so redrafting the bank never brings it back for you to reject twice.

This review is worth doing even if you never open Elicita during the meeting. A good question list makes you better prepared on its own, which is why it comes first and why you are meant to edit it rather than trust it.

If the bank comes back empty

The screen tells you which of these it was — it names the step that stopped and what to do about it. The four causes have four different fixes, and only one of them is waiting:

1. **Nothing set up** — drafting needs the AI provider from step 1.
2. **Nothing was read** — if every document you added was a *link* and your Microsoft 365 tenant is not registered, none of them were read. You would have seen each link refused at the time. Drop the files in instead.
3. **It is still working** — normal. Give it minutes, not seconds.
4. **A credential problem** — a key not permitted to use the model you chose, a provider throttling you, or a network it could not cross.

One kind of document contributes nothing either way: a **scanned PDF**, where the page is really a photograph of a page. Elicita will not guess at words in a picture, so it adds nothing rather than nonsense. If a scan is the only copy you have, put that document's important words in the vocabulary list instead.

Step 6 — Run the meeting

Consent first, and in this build it is yours to handle. As it ships today, Elicita does not stop to ask for consent before a meeting, and writes no consent record. The screen says so in as many words, and is careful to report that nothing has been recorded here rather than showing you a reassuring tick.

 BEFORE RECORDING

Discovery 3 — integrations and volumes

Consent model
standing for the engagement

Not required for this meeting
This engagement does not ask for consent at each meeting, so nothing is recorded here.
Not asked

WHAT HAPPENS TO THE AUDIO

Never saved as a file
Elicita writes no copy of the audio anywhere. It is held only for as long as it takes to turn it into text.

Destroyed after transcription
The moment the recording has been transcribed and diarized, the audio is deleted and the deletion is recorded.

Vendors told not to retain
Sent as a parameter on every request, not only in the contract.
On

NEXT

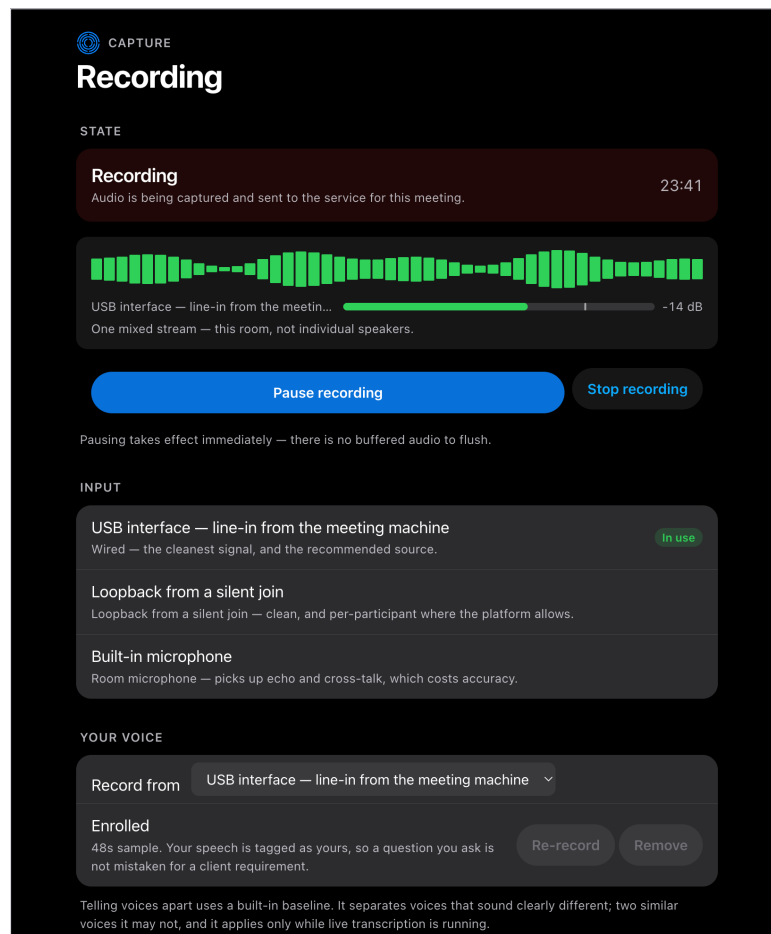
Set up the recording
Choose your microphone and check that sound is arriving, then start recording. The meeting begins when the recording does.
Continue to recording

Consent is not being asked for, and nothing is on record

So having the consent conversation — and being able to show later that you had it — rests with you, not with the software. The gate that would hold recording shut is built and works, but nothing can switch an engagement over to it yet.

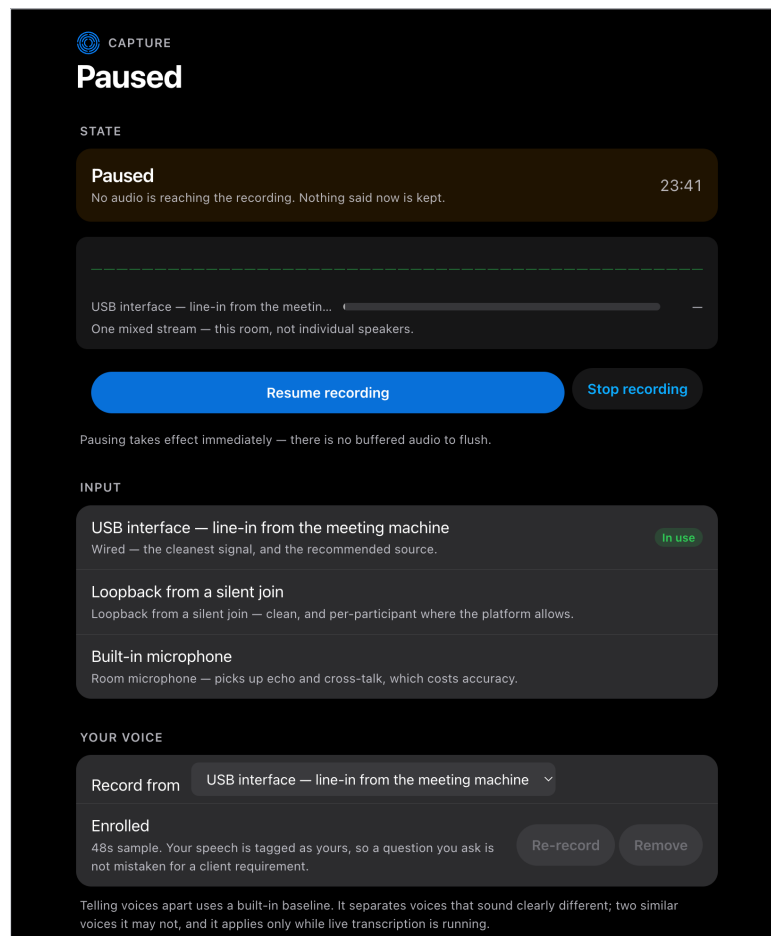
What you can tell the client about the audio, truthfully, before a second of it exists: Elicita writes no copy of it anywhere, it is held only for as long as it takes to turn it into text, it is destroyed the moment that finishes — and the deletion is itself recorded — and the transcription service is told with every request not to keep it.

Starting. Pick the input before you start, rather than discovering afterwards that it took the wrong one, and press *Check microphone* first: it opens the input and shows the level moving without recording anything, which is also what makes a browser tell Elicita the real names of your microphones. Starting the recording is what registers the meeting, and the input is opened before that happens, so a microphone that will not open registers nothing. Elicita will argue for a better microphone: a wired input or a silent join is markedly more accurate than a mic in the middle of the table, because people talking over each other causes more transcription errors than the choice of service does. A room mic earns a warning that stays on screen.



Recording, with pause one tap away

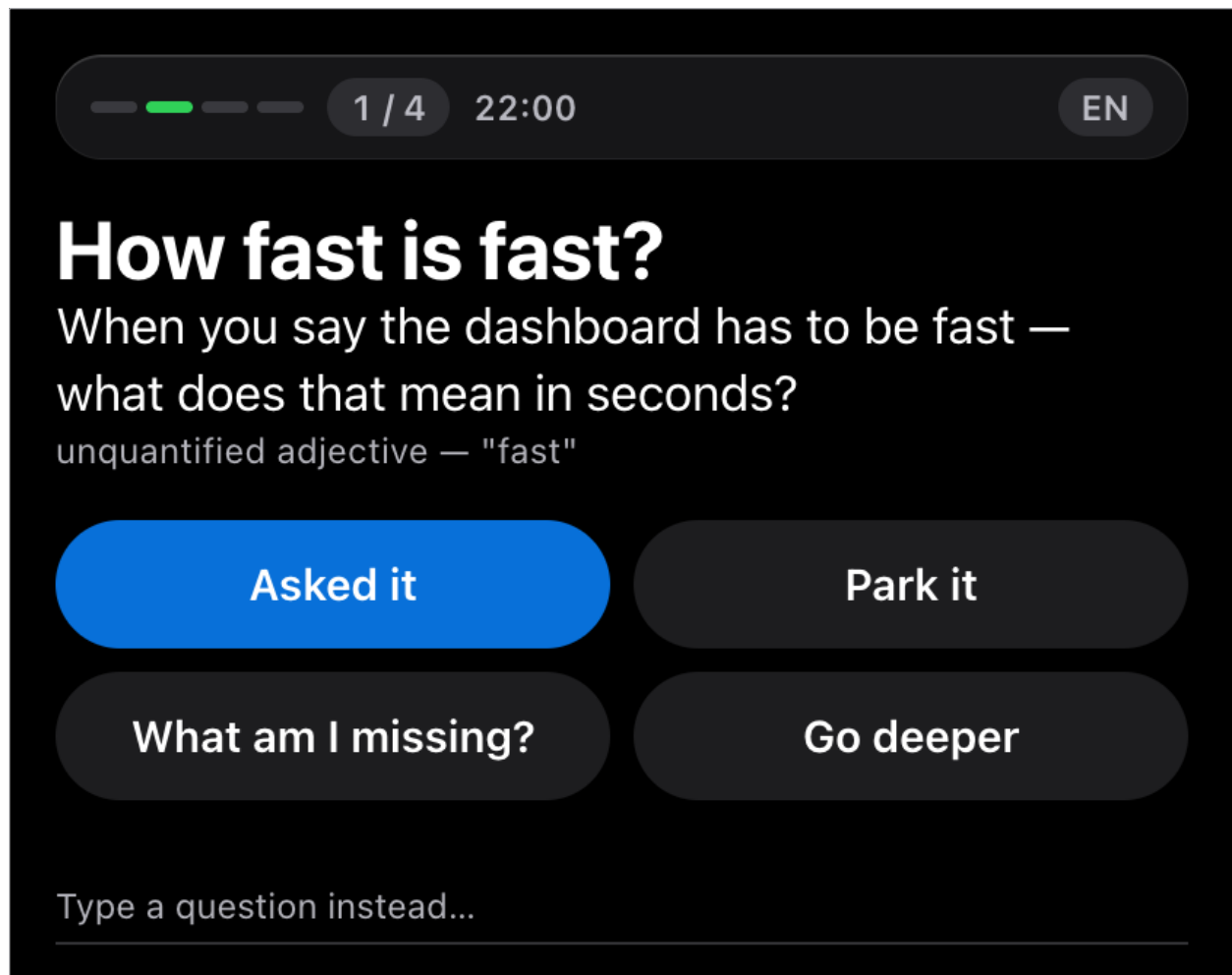
Pause is the control to know. You reach for it when a client says *"can we take this bit off the record."* It is the largest thing on the screen, never moves, takes effect immediately, and does not ask whether you are sure. Stopping sits beside it, deliberately smaller — it ends the recording and hands the microphone back, and is not what you want under your thumb when you meant to pause. Audio captured while paused is discarded, and resuming is instant.



Paused, and saying so unmistakably

The state is said three ways at once — the word, the sentence under it, and the colour — and never by colour alone, because believing you are paused while still recording is the worst thing this product could do to you.

The panel, when a suggestion arrives, gives you three things in the order you need them: the short version readable without breaking eye contact, the full wording if you want to ask it verbatim, and quietly underneath, *why it fired* — which is how you tell in half a second whether Elicita understood the room.

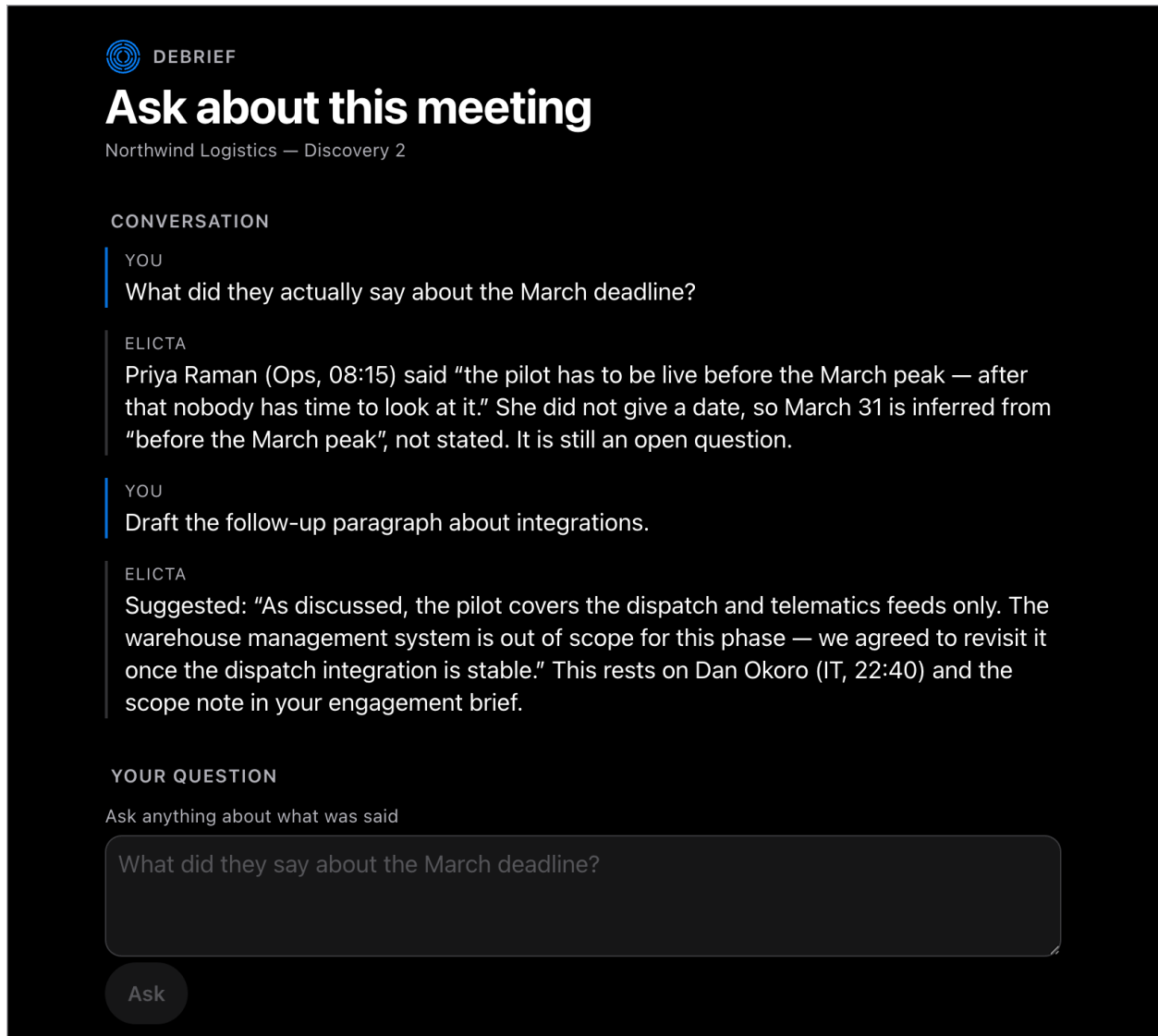


A suggested question, and why it fired

Every response is one tap: **Asked it** (the section fills in immediately), **Park it** (kept for the write-up), **Go deeper** (follow the thread), or **What am I missing?** (it names the biggest uncovered topic).

Step 7 — Afterwards

Open the debrief screen and **ask about the meeting in your own words** — *what did they actually say about the March deadline, draft me the paragraph about integrations*. Answers quote and attribute rather than summarise, and say plainly when something was worked out rather than said.



DEBRIEF

Ask about this meeting

Northwind Logistics — Discovery 2

CONVERSATION

YOU
What did they actually say about the March deadline?

ELICTA
Priya Raman (Ops, 08:15) said "the pilot has to be live before the March peak — after that nobody has time to look at it." She did not give a date, so March 31 is inferred from "before the March peak", not stated. It is still an open question.

YOU
Draft the follow-up paragraph about integrations.

ELICTA
Suggested: "As discussed, the pilot covers the dispatch and telematics feeds only. The warehouse management system is out of scope for this phase — we agreed to revisit it once the dispatch integration is stable." This rests on Dan Okoro (IT, 22:40) and the scope note in your engagement brief.

YOUR QUESTION
Ask anything about what was said

What did they say about the March deadline?

Ask

Asking a question about the meeting in your own words

The intended output is four documents: the open questions ranked by how much they matter, a log of decisions, a draft project brief, and a follow-up email for you to read and send — never sent for you. Every claim carries its source, and is labelled **Stated** (the client said it; check the wording) or **Inferred** (Elicita concluded it; judge whether you agree). A claim without a source cannot be saved at all.

If a step of the write-up fails, the sequence stops there and says which step and why, rather than passing invented material to the next one. A partial result with an explanation is recoverable; a complete-looking document with a fabricated section is not.

Into the next meeting, the open questions lead the screen, ahead of completed work — what you settled is behind you, and what you did not is what you are walking back in to deal with. A question that has come through three meetings unanswered is flagged, because there are only two explanations and both matter: it is genuinely hard, or it is being deflected.

What to expect on your first run — honestly

This build is not finished, and it is better to know where before your first client meeting than during it.

What is finished changes with every build, so this chapter deliberately does not list it. The last chapter of this

book does, and it is rewritten from the product's own notes each time the book is built — which is why it is the one to read before you promise anything to anyone. A list copied into this chapter would be out of date within a fortnight and would look just as confident.

The honest shape of your first real meeting, in a sentence: **Elicita is a very good prepared-question list, a recorder, and an after-the-fact way to interrogate what was said** — and, once a speech provider is configured, a panel that puts a question in front of you while the client is still in the room. That last part is the thing it is ultimately for, and it now works for vague wording. What it does not yet do is in the last chapter.

If something looks wrong

Symptom	First thing to check
A screen is empty	The client switcher at the top of the window. It is far more often pointed elsewhere than genuinely empty.
The question bank is empty	The screen names which of the four causes it was. Note that a failed compile's explanation does not survive restarting the service — press Compile again to get the current answer.
Pasting a document link is refused	Microsoft 365 registration is not set up. Drop the file in instead; that path needs nothing.
Start recording is greyed out	The screen says which: the page is not secure, the browser offers no microphone access, or the machine has no microphone attached.
No microphone in a browser	Browsers only hand it over on a secure page — on a plain address the capability is <i>absent</i> , not blocked, so there is no prompt to accept. That one is yours to fix and takes a moment.
A permission prompt during a client meeting	It should have been pushed out through device management, which grants microphone and screen-recording access centrally. Agree the endpoint-security exception <i>before</i> the pilot, not during it — it is the single most common cause of a bad first impression.

What Elicita deliberately will not do

Worth knowing on day one, because each is a choice rather than a gap:

- **It never speaks to the client.** No voice, no messages, nothing on your behalf. Ignore it for ninety minutes and nothing happens — that is a supported way to use it.
- **You cannot talk to it during a meeting.** Anything inviting you to type has taken your attention off the client, which is the exact problem it exists to relieve. Asking it things in your own words is an afterwards activity.
- **It stays quiet when unsure.** One suggestion that embarrasses you in front of a client does more damage than ten good ones repair, so it will sometimes miss what a sharp analyst would have caught. That trade was made knowingly.
- **It does not replace judgement.** It notices, suggests, and writes down what it heard. What matters, what to push on, and when the room has had enough stays with you.

The five-minute version

1. Settings -> first tab -> fill it in -> **Test**.
2. Add the client. Add a meeting to it.
3. Drop your documents on the Preparation screen. Tag them.

4. Type the client's product names and jargon into the vocabulary list.
 5. **Compile.** Go and make coffee.
 6. Read the drafted questions. Reorder. Prune.
 7. Walk into the meeting better prepared than you were — which is most of the value, and available today.
-

Where to read more

Everything here is covered again slowly in the chapters that follow, with every screen shown. The one to read next is the chapter on preparing for an engagement, because that is where the hour you spend pays for itself. The one to read before you promise anything to anyone is the last chapter, on what works today.

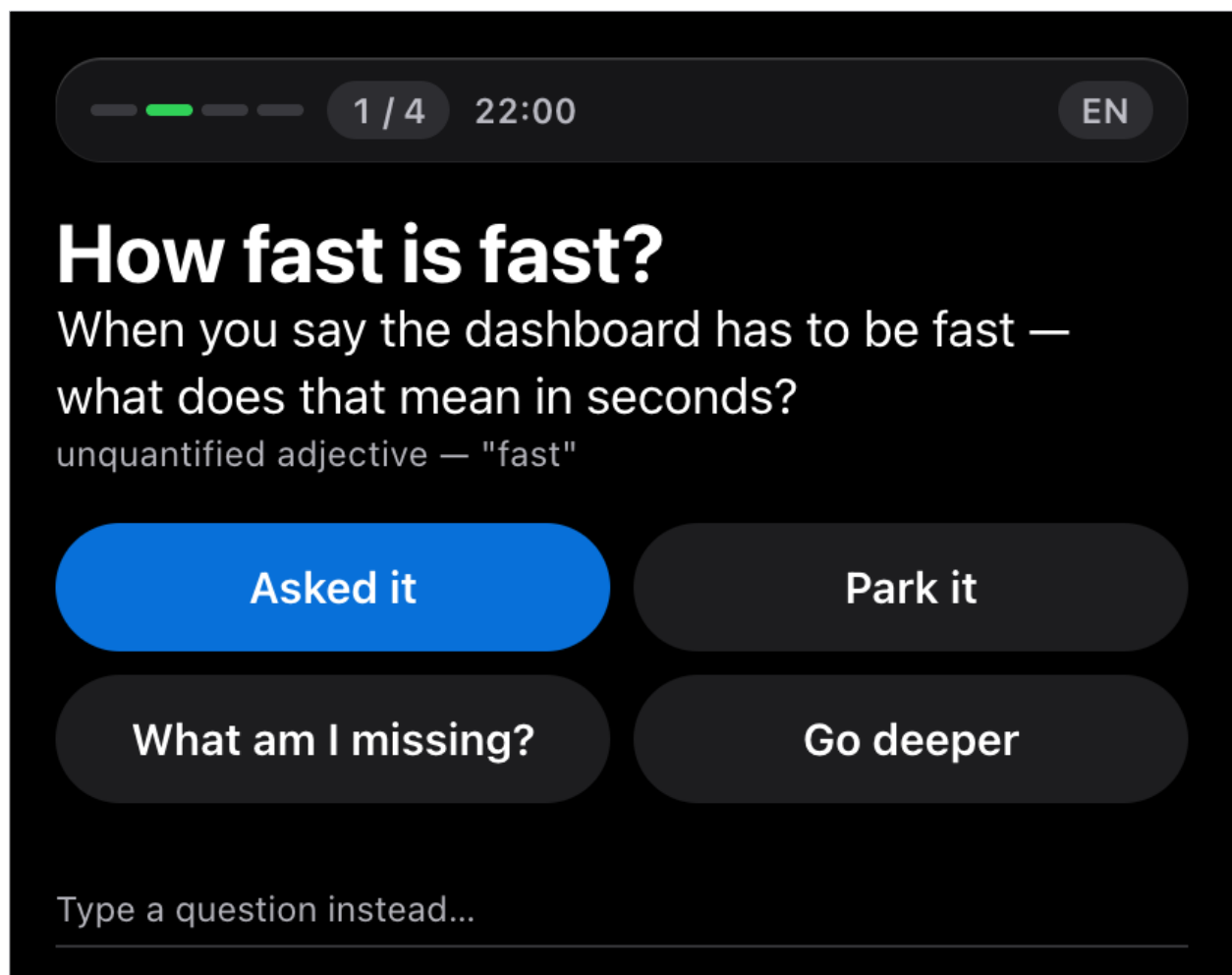
What Elicta Does

Imagine a meeting where a client is describing what they want built. Somebody says *"we need the reports to be fast."*

That sounds like a requirement. It is not one. Fast for whom? Compared to what they have now? A second, a minute, overnight? You almost certainly know to ask — and quite often do not, because at that moment you are also listening, taking notes, watching the clock, keeping the room comfortable, and trying to remember which topics you have not covered yet.

So the moment passes. "Fast" goes into the notes as though it meant something. Months later somebody builds a report that takes four seconds, the client says that is not what they asked for, and it gets built again.

Elicta watches for that moment and puts the question in front of you while there is still time to ask it.



A suggestion, with the short version you can read at a glance and the reason it fired underneath

The shape of it

What you do, and what Elicta does

1 Days before

You tell it about the client and give it what you already have. It drafts the questions worth asking. You read them, reorder them, and delete the ones that miss.

1

In the room

It listens. When something is left vague or a topic gets skipped, one suggested question appears. You decide whether to ask it, with a single tap.

2

Afterwards

It writes up what was agreed — the open questions, the decisions, a project brief, a follow-up email — with the words that were actually said attached to every claim.

The thinking is done in the first stage. The meeting only picks from work already finished.

What it is like to use

It is quiet. A good meeting might produce three suggestions, not twelve. When it is unsure, it says nothing, because one suggestion that embarrasses you in front of a client costs far more than a good one earns.

Every response is a single tap. There is nothing to type, nothing to read through, and no conversation to have with it. It offers; you accept or you do not.

And it never speaks to the client. It has no voice in the meeting, sends nothing on your behalf, and joins nothing as a participant that talks. If you ignore it for ninety minutes, nothing happens — that is a supported way to use it.

Who this guide is for

The person running the meeting, and whoever sets Elicita up for them. It covers what each screen does and how to use it, in order: setting it up, preparing for an engagement, the meeting itself, and what comes out afterwards.

It assumes nothing about how the software is built, and nothing about the company being interviewed. Where something is not finished yet, this guide says so rather than describing what is intended as though it already worked.

Setting It Up

Elicita uses outside services for two things: turning speech into text, and doing the writing-up afterwards. Somebody has to tell it which services to use and give it permission to use them. It is done in the app, once, by whoever administers Elicita for the team — not by an engineer editing a server somewhere, because the person who needs a key changed is rarely the person who could deploy one.

The screen is a row of tabs with one thing behind each, so you set up one service at a time instead of scrolling past five you did not come for. Most are an outside service; the rest are choices about your own installation, such as where the data is kept and whether every meeting asks about recording. The credential for a service sits behind that service's own tab, named after the company that issued it, so there is no list of keys to work out which of them goes where.

Before anything is set up

On first run the screen says plainly that nothing is configured, rather than looking ready and failing later. A tab whose service still needs something carries a small mark, so what is left to do can be read from the row of tabs before you open any of them; open one and it says which — this needs a credential, that needs a key.

SERVICE

Settings

These settings are held in memory and will be lost when the service restarts.

Claude • Speech • Recording Documents Storage

Claude Needs a credential

Drafts the question bank before the meeting and writes the debrief after it. Nothing during the meeting waits on it.

Route Claude calls through

Calls `api.anthropic.com` with the credential below.

Anthropic

Authenticate with

Whichever your organisation issues. Only the one selected here is sent, and only its field is shown.

API key OAuth token

Anthropic API key

A key issued from the Anthropic console (starts `sk-ant-`).

Not configured Test

No key is stored yet.

Model

Used for the context compiler and the debrief pipeline.

claude-opus-5

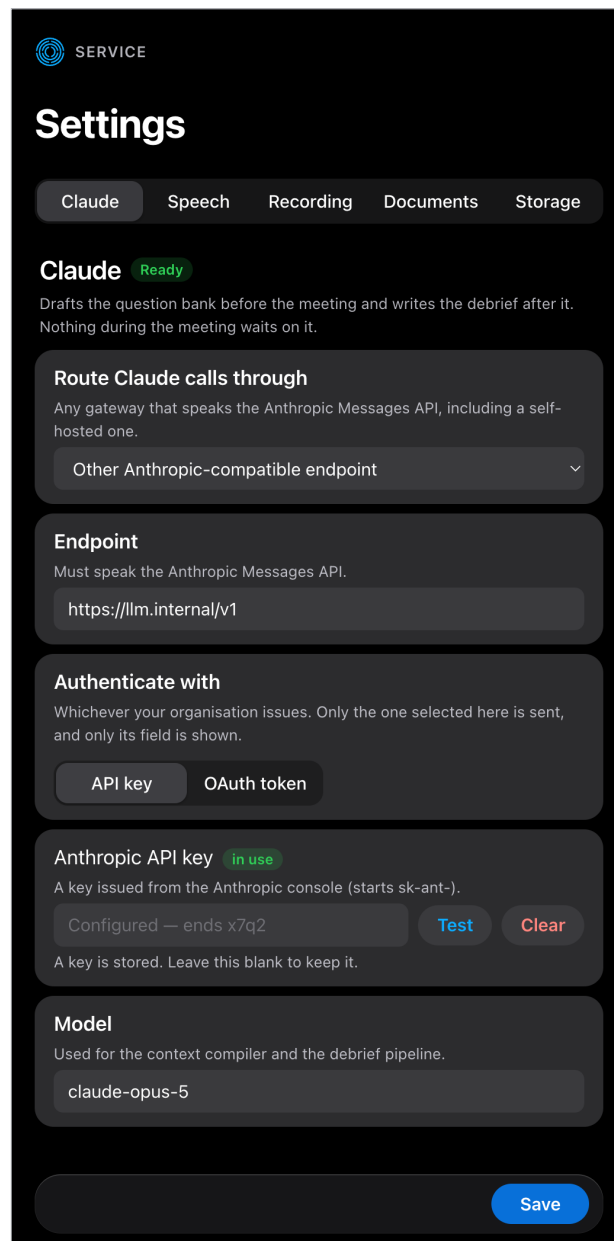
Save

Before anything is configured

Choosing where the writing-up happens

You can point Elicita at the company that makes the underlying service directly, at Amazon's, Google's or Microsoft's hosted versions of it, or at your organisation's own internal gateway. Each choice asks only for what it actually needs, and refuses to save if something is missing — a message here is better than a failure halfway through writing up a meeting.

Two of those choices ask for no key at all. If you already run on Amazon or Google, Elicita uses the permissions your cloud account grants it. Asking you to paste a key there would mean creating a long-lived credential in a place where your own platform has a better mechanism — so when you choose one of them, the credential fields do not appear at all rather than sitting there greyed out.



The screenshot shows the 'Settings' page for the 'SERVICE' (Claude). At the top, there are tabs for 'Claude', 'Speech', 'Recording', 'Documents', and 'Storage'. The 'Claude' tab is selected and marked 'Ready'. Below this, a description states: 'Drafts the question bank before the meeting and writes the debrief after it. Nothing during the meeting waits on it.' The settings are organized into sections: 'Route Claude calls through' with a dropdown menu set to 'Other Anthropic-compatible endpoint'; 'Endpoint' with a text field containing 'https://llm.internal/v1'; 'Authenticate with' with buttons for 'API key' and 'OAuth token'; 'Anthropic API key' with a status 'in use', a text field showing 'Configured — ends x7q2', and 'Test'/'Clear' buttons; and 'Model' with a text field containing 'claude-opus-5'. A 'Save' button is at the bottom right.

SERVICE

Settings

Claude Speech Recording Documents Storage

Claude Ready

Drafts the question bank before the meeting and writes the debrief after it. Nothing during the meeting waits on it.

Route Claude calls through

Any gateway that speaks the Anthropic Messages API, including a self-hosted one.

Other Anthropic-compatible endpoint

Endpoint

Must speak the Anthropic Messages API.

https://llm.internal/v1

Authenticate with

Whichever your organisation issues. Only the one selected here is sent, and only its field is shown.

API key OAuth token

Anthropic API key in use

A key issued from the Anthropic console (starts sk-ant-).

Configured — ends x7q2 Test Clear

A key is stored. Leave this blank to keep it.

Model

Used for the context compiler and the debrief pipeline.

claude-opus-5

Save

Pointing Elicita at an internal gateway

Keys and tokens

Both are supported, because organisations issue different things. You say which kind you have, and only that one is asked for — the screen no longer shows you two boxes and leaves you to work out which of them matters. If the other kind is still stored from before, Elicita says so in a line of its own and offers to remove it, so nothing is left on the service that you cannot see.

The screenshot shows the 'Settings' screen for the 'Claude' service. At the top, there's a 'SERVICE' header with a logo. Below it, the title 'Settings' is prominent. A horizontal menu contains tabs for 'Claude', 'Speech', 'Recording', 'Documents', and 'Storage', with 'Claude' being the active tab. The 'Claude' section is marked 'Ready' in green. A description states: 'Drafts the question bank before the meeting and writes the debrief after it. Nothing during the meeting waits on it.' Below this, the 'Route Claude calls through' section shows a dropdown menu set to 'Anthropic', with a note: 'Calls api.anthropic.com with the credential below.' The 'Authenticate with' section has two buttons: 'API key' (selected) and 'OAuth token'. The 'Anthropic API key' section is marked 'in use' in green. It includes a text field showing 'Configured — ends x7q2', a 'Test' button, and a 'Clear' button. A note below says: 'A key issued from the Anthropic console (starts sk-ant-). A key is stored. Leave this blank to keep it.' The 'Model' section has a text field containing 'claude-opus-5'. At the bottom right, there is a blue 'Save' button.

Credentials configured, with the one in use marked

Four details on this screen exist to stop a credential leaking, and they are worth knowing about because they change how the screen behaves:

Why the credential fields behave oddly

The field is always empty

Elicita never sends a stored key back to the screen, so there is nothing to show. It displays the last four characters instead, which is enough to tell one key from another.

Blank means leave it alone

Saving a different setting cannot wipe your key by accident.

Clear is separate and deliberate

Removing a credential is its own action, because it cannot be undone.

Test really tests

It checks the credential against the service rather than reporting "configured" as though that meant "working". If it fails you see the service's own explanation, never your key.

Speech keys

Transcription keys are a list you add to, not a field you fill in. You can hold as many as you have, from as many companies, and each one is listed under the name of the company that issued it — the name on the tab you copied it from.

Settings

Claude Speech Recording Documents Storage

Speech to text 2 keys

Turns the meeting into the transcript everything else reads. This is the credential the meeting itself depends on.

Speech keys

As many as you have, from as many providers. Changes here save themselves — the Save bar below is for the rest of this screen.

Deepgram Northwind **SERVING**
Serving the nudges and the recording both.
[Test](#) [Take out of service](#) [Remove](#)

AssemblyAI Northwind
Transcribes the recording after the meeting. This build has no live recogniser for this provider, so it does not drive the in-meeting nudges.
[Test](#) [Take out of service](#) [Remove](#)

Add a speech key

Paste it once. It is stored write-only — afterwards this screen can say a key is there and show its last four characters, and can never show you the key again.

Deepgram Which account Key

Add

When there is more than one

One key serves every request until you change it. The others stay as spares.

Use one key

Recording transcription

Two engines transcribe the recording separately after the meeting; where they disagree is flagged for you to check. They must be different vendors — the same engine twice would always agree with itself.

Deepgram + AssemblyAI

Speech region

Where audio is processed. Pin it to the region your engagement requires; closer regions also respond faster.

eu

☒ **Send engagement vocabulary to the transcriber**
Client and product names are transcribed far more accurately when the engine is told about them in advance.

☒ **Tell vendors not to retain client audio**
Sets the opt-out on every request. Your contract may already say this; sending it per request is what an audit can verify.

Save

Speech keys

There is no separate setting for which company transcribes. Adding a key is choosing one. That is why a list replaced the single field that used to be here: with one field, a second key silently replaced the first, and the only sign was four characters changing.

Each key says what it is actually doing, because the two jobs do not want the same companies. One transcribes while people are still talking, which is what the suggestions during the meeting depend on; the

other transcribes the recording afterwards, and needs two different companies so that where they disagree can be flagged for you. A key that does one and not the other says so on its own line, rather than leaving you to wonder whether it is broken.

Everything in this list saves itself the moment you do it. The Save bar at the bottom of the screen is for the rest of the settings, not for these.

Three things you can do to a key you already have. **Test** asks the company itself, and you see their answer rather than ours. **Take out of service** stops it being used and keeps it, which is what you want while you find out whether a key has been revoked. **Remove** asks once and then it is gone — the key itself is never shown again, by design, so there is nothing to copy back.

When you hold more than one, the setting below the list decides how they are used: one key until you say otherwise, or each request taking the next in turn, which spreads a long meeting across several accounts instead of exhausting one.

You can also point Elicita at a service it does not ship support for, though it will tell you that some behaviour then needs checking against that provider yourself.

Reading documents from SharePoint or OneDrive

This one is optional, and it is worth knowing exactly what it buys you before you go asking anybody for it.

Documents reach Elicita two ways: you drop the files onto the preparation screen, or you paste a link to them. Dropping files needs nothing set up at all. A link has to be fetched, and fetching from your organisation's SharePoint or OneDrive means Elicita has to be allowed in — which is a registration your Microsoft administrator creates, giving you a directory id, an application id and a secret. Those three go here.

Until they are filled in, pasting a link is refused, with a message saying so. That is deliberate. A link recorded but never read would sit in the document list looking like preparation that had happened, and the questions drafted from it would quietly be drafted from nothing.

Where your data is kept

Everything an engagement remembers — the client, its meetings, the documents you attach and what was read out of them, the words you add, and the drafted questions — is written to a single file on this machine as you work. Nothing needs installing for that, and nothing leaves the machine to make it happen.

A firm that would rather keep all of it on its own database server can say so here instead, and Elicita will use that from then on. The connection details include a password, so they are stored the same way as every other credential on this screen: entered once, never shown back. What the screen does show is which server is in use, with the password taken out, so you can confirm at a glance that a machine is pointed where you think it is.

A change here takes effect the next time the service starts, not the moment you save it. The screen says so rather than letting you assume otherwise.

Whether every meeting asks about recording

Elicita can work either of two ways, and which one it uses is set here rather than decided meeting by meeting.

As it arrives, consent is treated as standing for the engagement: no meeting stops to ask, and recording begins the moment you start it. That suits a firm whose engagement letter already covers recording, and it is the quieter of the two. It also means nothing is written down — a recording made this way carries no record that anyone was told, because none was ever taken.

The other setting makes every meeting pause before it records, show the wording you would read out, and keep a note of who confirmed it and when. That note is what an audit later asks for.

Which is right is a question about how your engagements are contracted, and Elicita cannot answer it for you.

What it can do is stop the choice from being invisible, which is why it sits on this screen with the consequence written next to it rather than being buried where only a developer would find it.

For a shared installation

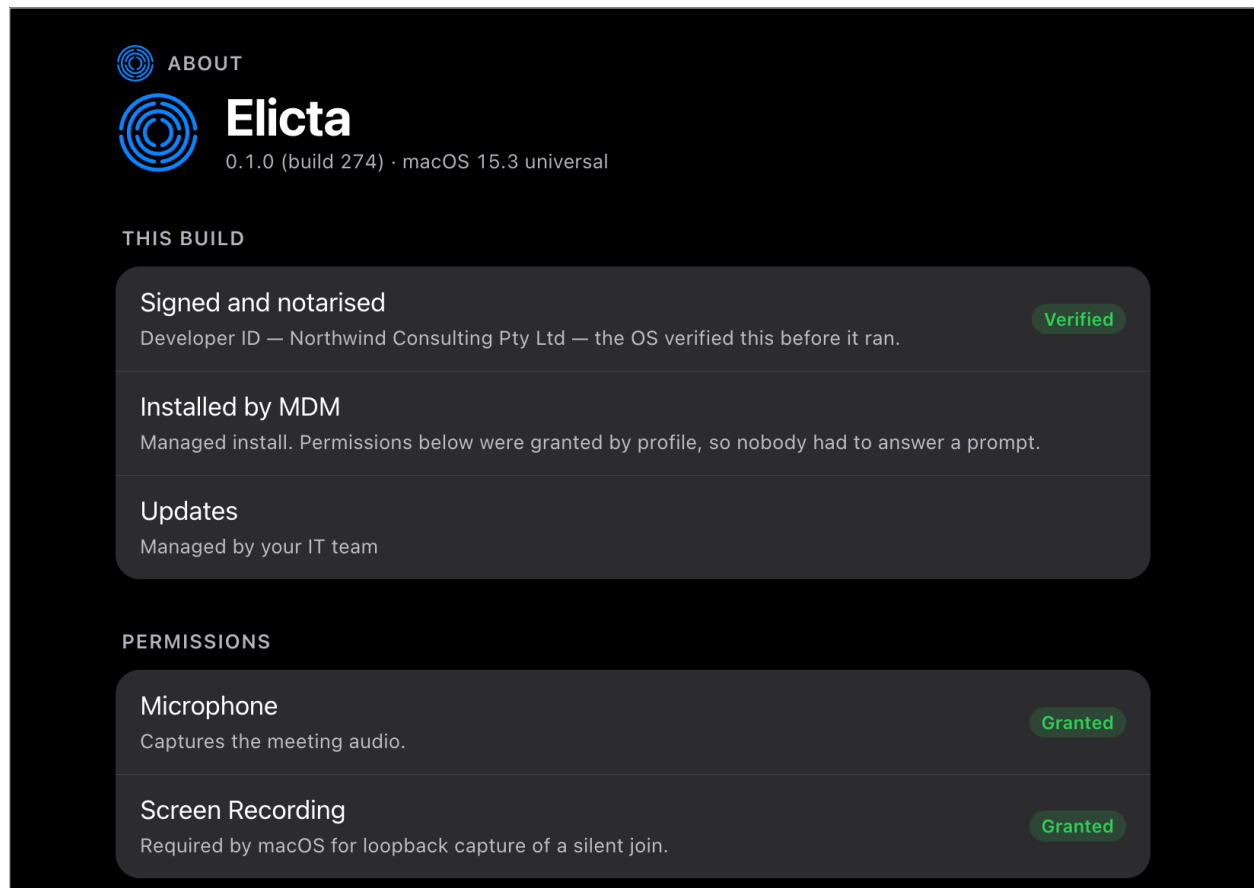
Where several machines share a deployment, the key can come from your own secret store rather than being typed in — anything with a command line will do. If the store cannot be reached, Elicta refuses to start and says why, rather than quietly generating a new key that would leave the other machines unable to read anything.

Getting It Onto People's Machines

This is invisible when it goes well and very visible when it does not. A meeting tool that asks every person for permission to record their screen generates the same support ticket over and over.

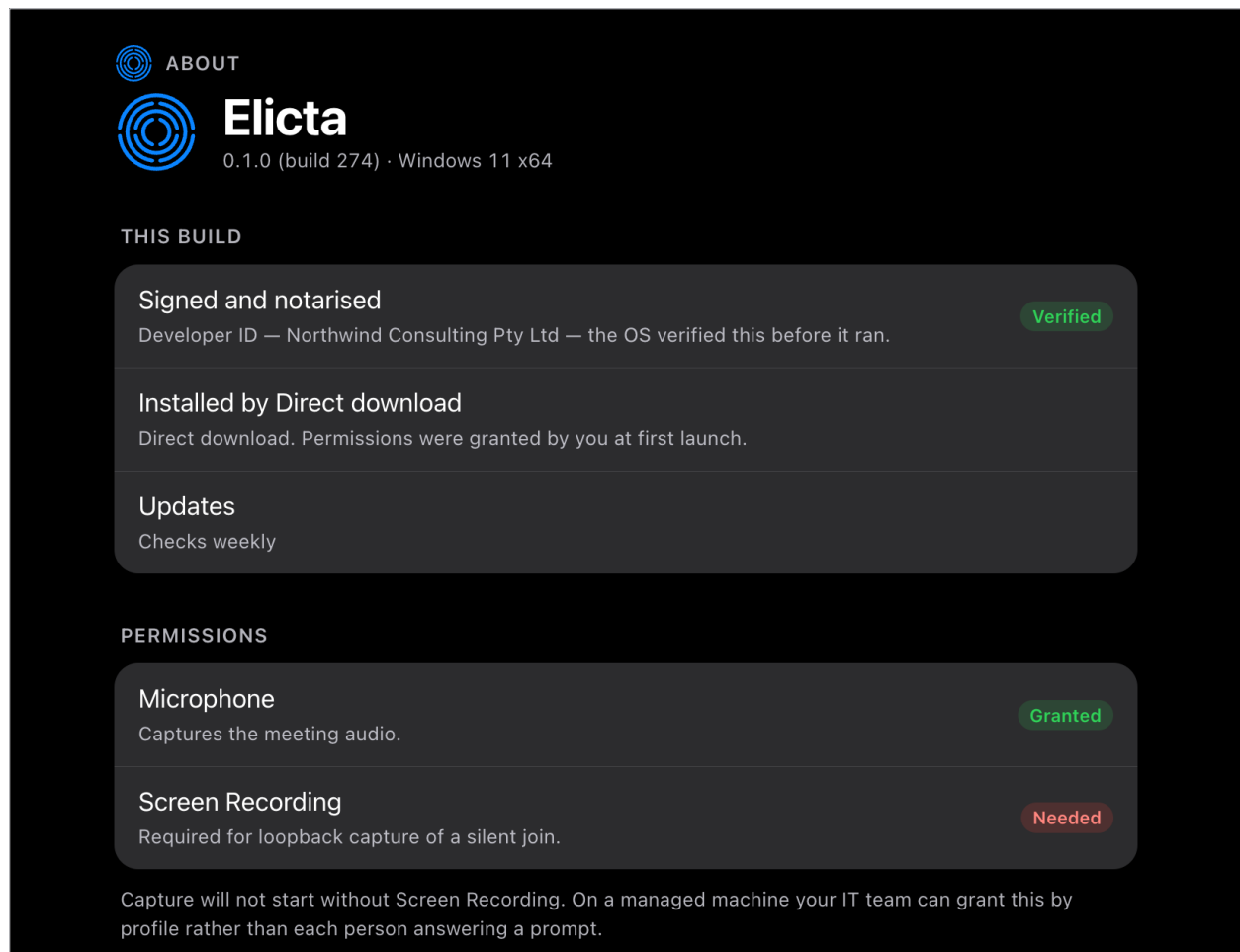
The preferred route

Push Elicta out through your device management system. That is the recommended route rather than the fallback, because it lets you grant microphone and screen-recording access centrally. Nobody sees a permission prompt, and nobody has to be talked through one during a client meeting.



A managed installation, showing what was granted centrally

Keep the plain installers as a backup for machines outside management.



An unmanaged installation

Agree the security exception first

An unfamiliar application opening an audio stream is exactly what endpoint security software is built to flag. It will flag it — and if this is left until the pilot, it will flag it in the middle of somebody's client meeting.

Get the exception agreed before the pilot rather than during it. This is the single most common cause of a bad first impression.

What each person does at first launch

Very little, if the push worked. Otherwise they will be asked once for microphone access and, on some systems, for screen recording. Both are required for Elicta to hear the meeting at all.

BEFORE A MEETING

Preparing for an Engagement

Almost everything that makes Elicita useful happens before anyone sits down. That is deliberate: the thinking is done in advance so that during the meeting it only has to pick the right question from work already finished. It is the only way a suggestion can arrive while it is still worth having.

Do this days before the first meeting, not the morning of it.

Tell it about the client

Elicita opens on your clients. Each one is an engagement, and adding one asks for three things: who they are, the sector, and the commercial shape of the work.

That is not filing. It is what lets Elicita work out which languages to expect in the room, so that nobody has to be asked to choose one before a meeting starts.

ENGAGEMENTS

Your clients

ENGAGEMENTS

Northwind Logistics
Freight and logistics · Fixed-price discovery, three meetings [Remove](#)

Calder & Rowe
Professional services · Matter-management replacement [Remove](#)

Opening one makes it the client every other screen is about. Removing one takes it off this list along with its meetings, documents and vocabulary — nothing is erased, and it can be brought back.

NEW ENGAGEMENT

Client organisation
Northwind Logistics

Sector
Freight and logistics

Commercial context
Fixed-price discovery, three meetings

The sector and the commercial shape are not filing. They are what lets Elicita work out which languages to expect in the room. All three are needed before an engagement can be created.

Create engagement

Your clients, and the form for adding one

Opening a client is what makes it the one every other screen is about, and the row itself is what opens it —

press anywhere along it. The one in force is tinted rather than labelled, the way a chosen row looks anywhere else on this machine.

It is also where you remove a client you no longer want on the list, which takes its meetings, documents and vocabulary out of view with it and erases nothing. Removing asks first, in a dialog naming the client and saying what removal actually does — one press taking a whole client off every screen at once was more than a single button's worth of consequence. Removing a meeting asks in the same way, and says what it keeps.

The safe answer is the one already selected, so pressing Enter out of habit keeps rather than removes, and Escape is a way out rather than agreement.

Everything else on this page happens on the Preparation screen, about the one client you opened.

Switching between them

Once a client is open, the quickest way to a different one is the control at the top of the window, beside the name of the screen you are on. It follows you from screen to screen, so you do not have to come back to the list to change your mind.

Your choice is remembered, so reopening Elicita puts you back where you left off rather than on whichever client happens to be oldest. And a screen that looks empty is far more often pointed at a different client than genuinely empty — that control is the first place to check.

Set up the meetings it will have

An engagement is a client. A meeting is one conversation with them, and everything after this page is about a meeting — consent, the live panel, the recording, the write-up. So an engagement needs at least one before any of that applies to it.

Adding one asks a single question: how the audio will reach Elicita. Line-in from the meeting machine, a silent join that takes the audio as loopback, or a microphone. You can change that later.

A new meeting arrives without a name, so the list shows it by its number. Give it one with Rename — "Discovery 2 — depot volumes" reads better than a number, and by the third meeting under one client it is the difference between a list you can use and three identical rows. Remove takes a meeting off the list. Nothing is erased by it: if the meeting happened, its consent record and its recording stay exactly where they are. It is there for the meeting you added by mistake.

Which engagement and which meeting you are working on is chosen at the top of the window, and the choice follows you from screen to screen.

Add what you already have

Scoping decks, throughput studies, the last proposal. There are two ways in, and they are equally good:

Drop the files straight onto the preparation screen, or pick them with the button beside the drop area. Nothing needs setting up first, and Elicita reads what is inside them — Word, PowerPoint, Excel and PDF alike, not just the filename on the front.

Or attach a document by its SharePoint, OneDrive or Teams link. Those are the only kinds of link Elicita accepts, so that what it works from is the copy your client can also see rather than one that has quietly drifted onto somebody's laptop. Reading from your Microsoft 365 tenant is the one thing here that has to be set up first, and if it has not been, attaching a link is refused with a message saying so — rather than the document sitting in the list looking as though it counted.

Each document is tagged as one of three things, and the tag changes how Elicita treats it:

The three document tags

Ground truth	Established fact. If a client contradicts it, that is worth interrupting for.
Hypothesis	Our assumption, not theirs. It produces questions to verify rather than contradictions to flag.
Superseded	Kept for background, never used to challenge anyone.

The tag decides whether a contradiction is worth interrupting a meeting for.

Add the client's own words

Product names, internal systems, their own jargon — the words a transcription service would have no way of knowing.

This is the single most effective thing you can do for accuracy, and it is worth more than any other preparation on this page. A misheard product name does not just look sloppy. It reads to Elicita as something brand new, and it will interrupt you to ask about a system that does not exist.

Taking something back out

Every document has a **Remove** beside it, and every word has a small × on its chip. There is also a way to remove the whole engagement, at the foot of the page.

None of it erases anything. What you remove stops appearing — the engagement goes off the list and takes its documents and vocabulary out of view with it — and the record is kept, so a removal made by mistake is not a catastrophe.

That is worth being plain about in both directions. It means you can tidy freely. It also means removing a client here is *not* how you would honour a request to erase their data; that is a different thing, and Elicita does not do it yet.

The word list is the one where removing matters most. A wrong word handed to the transcription service is worse than no word at all, because it makes the transcript confidently wrong rather than merely uncertain — so a typo there is worth correcting the moment you notice it.

Review the questions before anybody asks them

Elicita reads everything and drafts the questions worth asking, grouped by the section of your requirements template that each one serves. You read them, move the ones that matter up the list, and prune the ones that miss.

A drafted bank runs to seventy-odd questions, so the sections open one at a time rather than arriving as one very long page, and each says how many questions are inside it before you open it. There is also a box that filters the whole bank by what a question says — the way to find the four that mention one system without reading the other seventy. While that filter is on you can still prune, but not reorder: moving a question up means moving it past the one above it, and under a filter the question above on screen is not the one above in the bank.

Pruning is not deletion. A pruned question is kept as a record of your judgement and left out of the list, so redrafting the bank later does not bring it back for you to reject a second time.

While you are reading a section, its heading stays at the top of the screen rather than scrolling away above the questions, so you can always see which part of the template the ones in front of you belong to.

Finding your way around a long page

With a full bank open this screen runs to a dozen screenfuls, and the meetings sit at the very bottom of it. Beside the page — across the top of it, in a narrow window — is a list of its parts: the documents, the words, the questions and the meetings, with the sections of the bank named underneath and the number of questions in each. It marks the part you are reading as you move, and takes you to any other in a single click. That is the short way to the foot of the page without crossing the questions to get there, and the short way back.

Clicking a section of the bank opens it as well as going to it, so you never arrive at a heading with nothing under it.

The keyboard works on this page too. Page Down, End and the arrow keys move it once you have clicked anywhere on the page, which on a screen this long is worth knowing.

The screenshot shows the Elicita preparation interface for 'Northwind Logistics'. At the top, a navigation bar includes 'Reference documents', 'Engagement vocabulary', 'Question bank', and 'Meetings'. The main content is divided into four sections:

- REFERENCE DOCUMENTS:** A list of documents with their status and actions.

Document	Status	Action
Current-state scoping deck	ground truth	Remove
Warehouse throughput study 2025	ground truth	Remove
Proposed integration approach	hypothesis	Remove
Original RFP response	superseded	Remove

 Below this is a 'Drop documents here' area with a 'Choose files' button.
- ENGAGEMENT VOCABULARY:** A section for adding terms. It shows 'Zephyr WMS' as a term added to the 'Product name' category. A note explains that removed terms are kept and can be brought back.
- QUESTION BANK:** A section for managing questions. It shows a list of questions grouped by section:
 - Performance (2 questions):**
 - When you say the dashboard has to be fast, what does that mean in seconds? (Priority 1 - 01-scoping-deck.pptx)
 - Is that response time at median load, or at your Monday morning peak? (Priority 2 - 02-throughput-study.xlsx)
 - Integrations (2 questions)**
 - Volumes (1 question)**
- MEETINGS:** A section for managing meetings. It shows a meeting titled 'Discovery 3 — integrations and volumes' with a 'Line-in from the meeting machine - scheduled' status. A note explains that the meeting record stays where it is.

The preparation screen: documents, the client's vocabulary, and the question bank grouped by section

Under each question is where it came from: the document it was drafted from and how you tagged that document, or *reasoned from the engagement* where there was no page behind it and Elicita worked it out from

the sector and the commercial shape instead.

That line is the one to read when you are deciding what to cut. A question drawn from a document you called ground truth stands on something the client wrote down. A reasoned one is a guess — often a good guess, and on a thin set of documents most of the bank will be guesses, which is worth knowing before you walk in rather than after. It also makes a long bank quicker to review: you can take out the guesses in one pass and read the rest properly.

This review is worth doing even if you never open Elicita during the meeting. A good question list makes you better prepared on its own — which is why it comes first, and why you are meant to edit it rather than trust it.

If the question bank comes back empty

Three things can cause it, and it is worth knowing which.

The first is setup: drafting the questions needs one of the outside services configured, which is covered earlier in this guide.

The second is that a draft can only work from documents it has actually read. If every document you added is a link and your Microsoft 365 tenant is not registered yet, none of them were read — and you would have seen each link refused at the time, with a message saying so. Dropping the files in instead needs nothing configured, and is the quickest way to get a real bank out of a first engagement.

The third is simply that it is not finished yet. Drafting a bank is slow work, so it is handed off as a job that takes some minutes; Elicita goes back for the result on its own and fills the bank when it arrives. Compile before you make coffee rather than while the client is waiting. While it is working the screen says so, and will not let you start a second one on top of it.

The screen tells you which of these it was. Where a compile ran and stopped, it names the step that stopped and what to do about it — nothing set up, a credential that is not allowed to use the model you chose, a service being throttled, a network it could not cross. Those have different fixes and only one of them is waiting. It used to say "not compiled yet" however many times a compile had run and failed, and the explanation went to a log the person reading the screen was never going to see.

Drafting the questions is normally handed to the cheaper of two ways of asking, which some accounts are not permitted to use. Where yours is not, Elicita asks the ordinary way instead rather than giving up — it costs more per compile and finishes in one go rather than two, which is the better trade when the alternative is no questions at all.

One kind of document contributes nothing either way: a scanned PDF, where the page is really a photograph of a page. Elicita does not try to guess at the words in a picture, so such a document adds nothing rather than adding nonsense. If a scan is the only copy you have, the vocabulary list is where that document's important words are best put.

Starting a Meeting

Recording a client without a clear record of their consent is the kind of mistake that ends an engagement. Elicita has a gate for exactly that — and in this build, the gate is set not to ask.

What this build does

As it ships today, Elicita does not stop to ask for consent before a meeting. Every engagement is treated as having settled consent once, for the engagement as a whole, so no meeting puts a question in front of you and recording can start straight away.

The screen says so in as many words, and it is careful about the difference between not asking and having an answer: it reports that nothing has been recorded here, rather than showing you a reassuring tick.

The screenshot shows a dark-themed interface with the following sections:

- BEFORE RECORDING** (header with a circular icon)
- Discovery 3 — integrations and volumes** (main title)
- CONSENT** (section header)
 - Consent model**
standing for the engagement
 - Not required for this meeting**
This engagement does not ask for consent at each meeting, so nothing is recorded here. Not asked
- WHAT HAPPENS TO THE AUDIO** (section header)
 - Never saved as a file**
Elicita writes no copy of the audio anywhere. It is held only for as long as it takes to turn it into text.
 - Destroyed after transcription**
The moment the recording has been transcribed and diarized, the audio is deleted and the deletion is recorded.
 - Vendors told not to retain**
Sent as a parameter on every request, not only in the contract. On
- NEXT** (section header)
 - Set up the recording**
Choose your microphone and check that sound is arriving, then start recording. The meeting begins when the recording does. Continue to recording

Consent is not being asked for, and nothing is on record

That is a deliberate setting for this stage, not something we overlooked. What it means in practice is that having the consent conversation, and being able to show later that you had it, rests with you and not with the software. Elicita will not stop you, and it will not be holding a record you can point to afterwards.

When consent is asked at every meeting

The gate itself is built and works. Where an engagement is set to ask at every meeting, recording cannot start until somebody confirms it on the record. The button is simply unavailable.

The screenshot shows a dark-themed interface titled "BEFORE RECORDING" with a sub-header "Discovery 3 — integrations and volumes".

CONSENT

Consent model
per meeting

Not confirmed
Capture cannot start until someone confirms this on the record. Required

RECORDING CONSENT REQUIRED

This meeting will be recorded and transcribed. Continuing confirms that every participant has been informed and has consented to being recorded.
NSW Surveillance Devices Act — all-party consent

Who is confirming consent

Name and role Confirm consent

This records who is accountable for having disclosed the recording — not the participants. It is kept for the life of the engagement.

WHAT HAPPENS TO THE AUDIO

Never saved as a file
Elicita writes no copy of the audio anywhere. It is held only for as long as it takes to turn it into text.

Destroyed after transcription
The moment the recording has been transcribed and diarized, the audio is deleted and the deletion is recorded.

Vendors told not to retain
Sent as a parameter on every request, not only in the contract. On

NEXT

Set up the recording
Confirm consent above, and this opens. Continue to recording

Consent not yet confirmed, and recording cannot start

Confirming happens here, on the same screen. Elicita sets out what has to be true before recording — that everyone in the meeting has been told and has agreed — together with the law that requirement rests on, and asks for the name of the person confirming it. That name is not a participant list. It identifies whoever is accountable for having disclosed the recording, which is what somebody reviewing this later actually needs to know.

Once confirmed, the screen keeps who confirmed it and when. The answer to "did we have permission for this?" is never a memory test.

 BEFORE RECORDING

Discovery 3 — integrations and volumes

Consent model

per meeting

Confirmed

Priya Raman (delivery lead) · 18 Aug 2026, 09:58

On record

WHAT HAPPENS TO THE AUDIO

Never saved as a file

Elicita writes no copy of the audio anywhere. It is held only for as long as it takes to turn it into text.

Destroyed after transcription

The moment the recording has been transcribed and diarized, the audio is deleted and the deletion is recorded.

Vendors told not to retain

Sent as a parameter on every request, not only in the contract.

On

NEXT

Set up the recording

Choose your microphone and check that sound is arriving, then start recording. The meeting begins when the recording does.

Continue to recording

Consent confirmed, and the way to the recording open

Choosing which way an engagement works is not yet something the screens can do, so for now every engagement takes the setting described above.

What happens to the audio, said before it happens

The same screen tells you what Elicita will do with the recording, before a second of it exists. You can read it aloud to a client who asks.

What happens to the recording

- 1 It is never saved as a file**
 Elicita writes no copy of the audio anywhere. It is held only for as long as it takes to turn it into text.
- 2 It is destroyed as soon as it is used**
 The moment transcription and speaker identification finish, the recording is deleted — and the deletion is itself recorded, so the destruction can be shown rather than asserted.

1 The transcription service is told not to keep it

Every request carries that instruction. Your contract may already say so; sending it with each request is what makes it something an audit can check.

Each of these is something Elicita does, not something it promises.

Leaving for the recording

Consent is a gate, and a gate's job ends when it opens. Once consent is settled this screen offers one thing — *Continue to recording* — and that is all it does. It starts nothing and it opens no microphone. Until consent is settled, the button is unavailable.

Nothing is booked here either. The meeting is registered at the moment the recording begins, on the next screen, so a meeting you set up and then abandon leaves nothing behind.

Checking the microphone before you record

The recording screen is where a meeting actually begins, and it gives you a step before it: **Check microphone**. It opens the input and shows you the level moving, and records nothing at all.

Getting to a recording

1 Check the microphone

Elicita opens the input you chose and shows a live level. Say something and watch it move. Nothing is being recorded, and the screen says so.

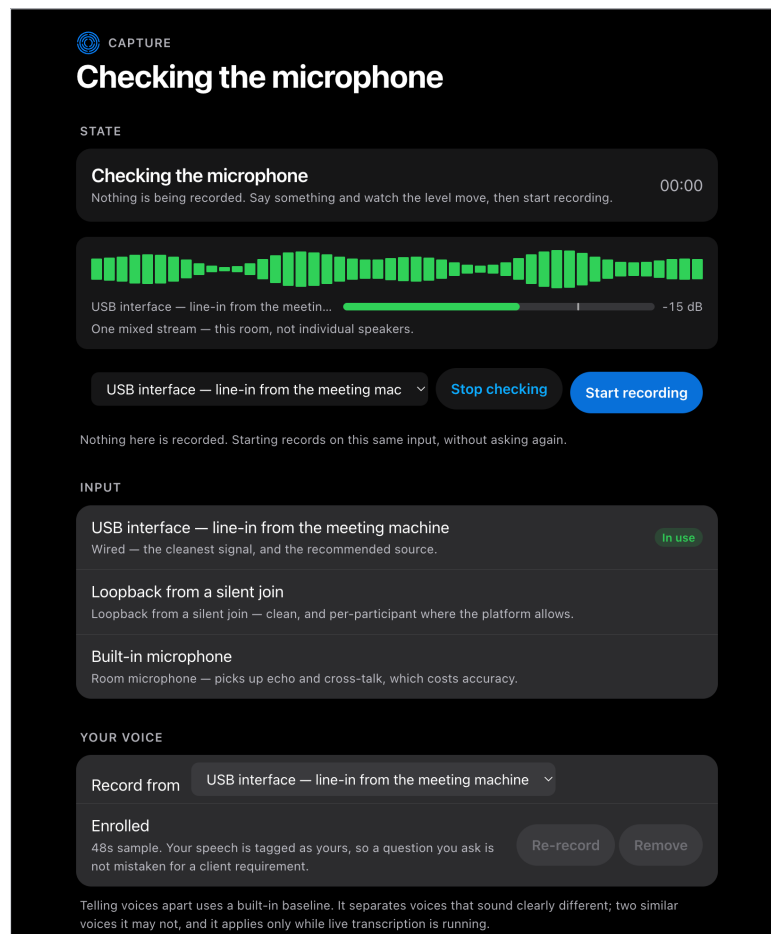
2 Choose the right input

Until you have checked one, a browser will not tell Elicita the names of your microphones, so the list reads "Microphone 1" and similar. Checking is what fills the real names in.

3 Start recording

The meeting is registered and recording begins, on the input you were just listening to. There is no second permission prompt and no gap.

The first is what turns the second into a real choice.



Checking the microphone, with nothing being recorded

This step is worth taking, and not only on the first meeting. A muted or unplugged microphone produces a screen that looks exactly like a working one for the length of a meeting, and the failure is discovered in the transcript when it is far too late to do anything about it. The level bar is the only thing that tells the two apart.

It also matters because the choice sticks. Once a recording is running the input cannot be swapped — changing it means stopping and starting again, which leaves a hole in the recording. Choosing before you begin is the only cheap time to choose.

If the microphone cannot be opened at all — you declined the permission, or another application is holding it — no meeting is registered. The screen tells you what happened in words you can act on, and you can fix it and try again. Elicita does not register a meeting it cannot record.

If there is nothing on the machine that can record, the controls are unavailable before you press them and the screen says why. The two common reasons are that no microphone is connected, and that you have opened Elicita in a browser over an ordinary web address rather than a secure one — browsers only hand over a microphone on a secure page, so on a plain address there is nothing to permit and no prompt to accept.

Once it is running, the recording stays running. Moving between screens does not interrupt it, and neither does going back to the consent screen. It stops when you stop it.

Joining the meeting

Elicita can take audio from a wired input on the machine, or join the call as a silent participant that never speaks. The silent join is markedly more accurate, for a reason covered in the chapter on controlling the recording: people talking over each other causes more transcription errors than the choice of service does.

DURING THE MEETING

The Panel and the Suggestion

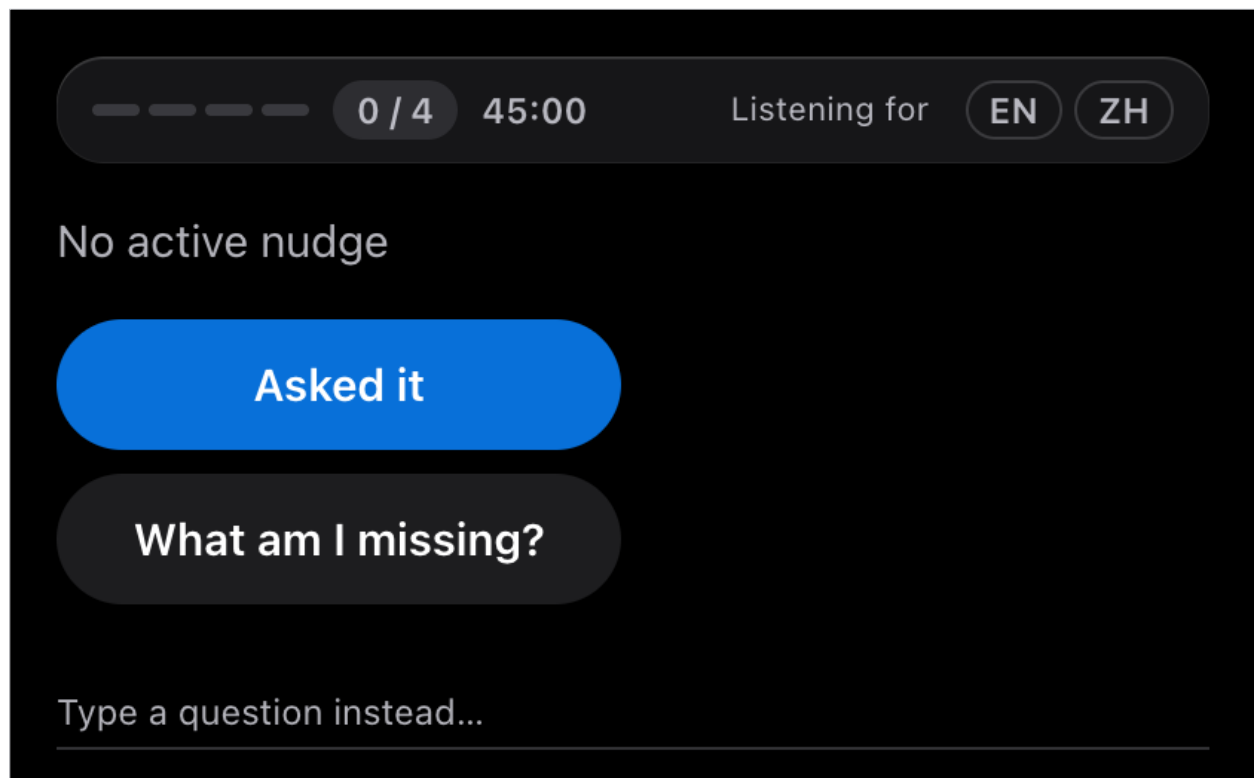
This is what Elicita is for. A client says something that sounds like an answer but is not one, and five seconds later you ask the question that pins it down — instead of thinking of it in the car afterwards.

Two things to know before the rest of this chapter describes it. What the microphone hears is now turned into words while the meeting runs, checked for the kind of vagueness that costs a requirement, and a question is put in front of you — so what follows describes what you will actually see, not what the screen would do if something reached it.

The checks that raise a question are the ones about wording: an amount with no number, an adjective with no threshold, a date that is not a date, a commitment behind a hedge. Those need no model and are quick enough to land while the sentence is still in the air. Three other ways a question could be earned are not built yet — an action with nobody named as doing it, an answer that contradicts a document you supplied, and a system or team nobody has mentioned before — so a meeting can pass without a suggestion and that is the product being quiet rather than broken.

Before anyone speaks

The panel shows how much of your template is covered, how long is left, and which languages it is listening for — worked out from what you told it about the client, so nobody has to pick one before the meeting starts. There is no suggestion, because nobody has said anything yet.

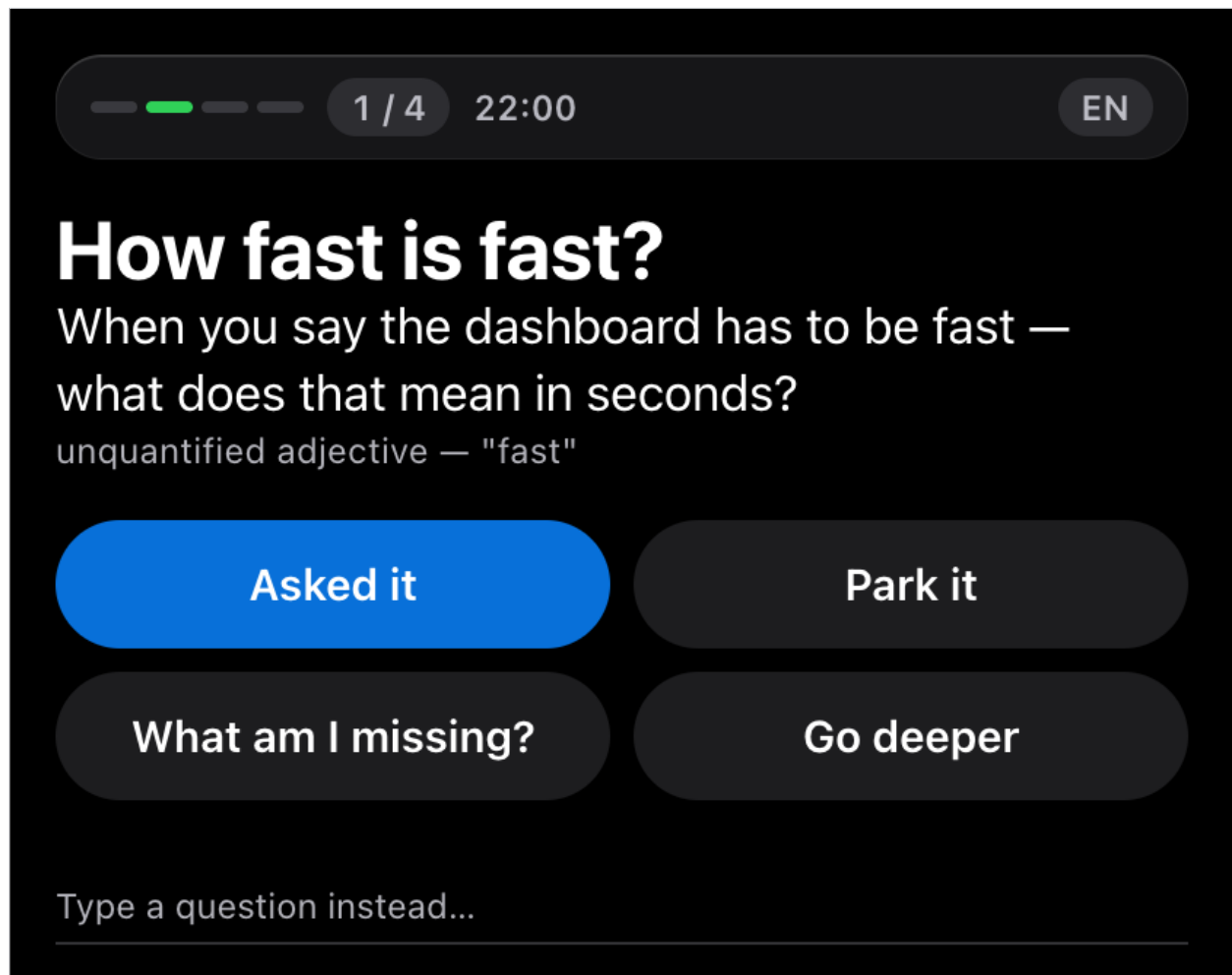


The panel before the meeting starts

A near-empty panel is the correct resting state. Anything more would pull your eyes to a screen during the part of the meeting where you are establishing rapport.

Something vague lands

The client says *"the dashboard has to be fast."* That is not a requirement, it is a feeling. Elicita recognises it and puts one question in front of you.



A suggested question, and why it fired

Three things, in the order you need them:

What is on a suggestion, and why

The short version	Readable at a glance, without breaking eye contact.
The full question	The exact wording, if you want to ask it word for word.
Why it fired	Quietly underneath. This matters more than it looks — it is how you tell in half a second whether Elicita understood the room or misheard it, and therefore whether to trust the next one.

Only one suggestion is shown at a time. Earlier ones move below a line and fade back — still readable if you want them, never competing with the current one.

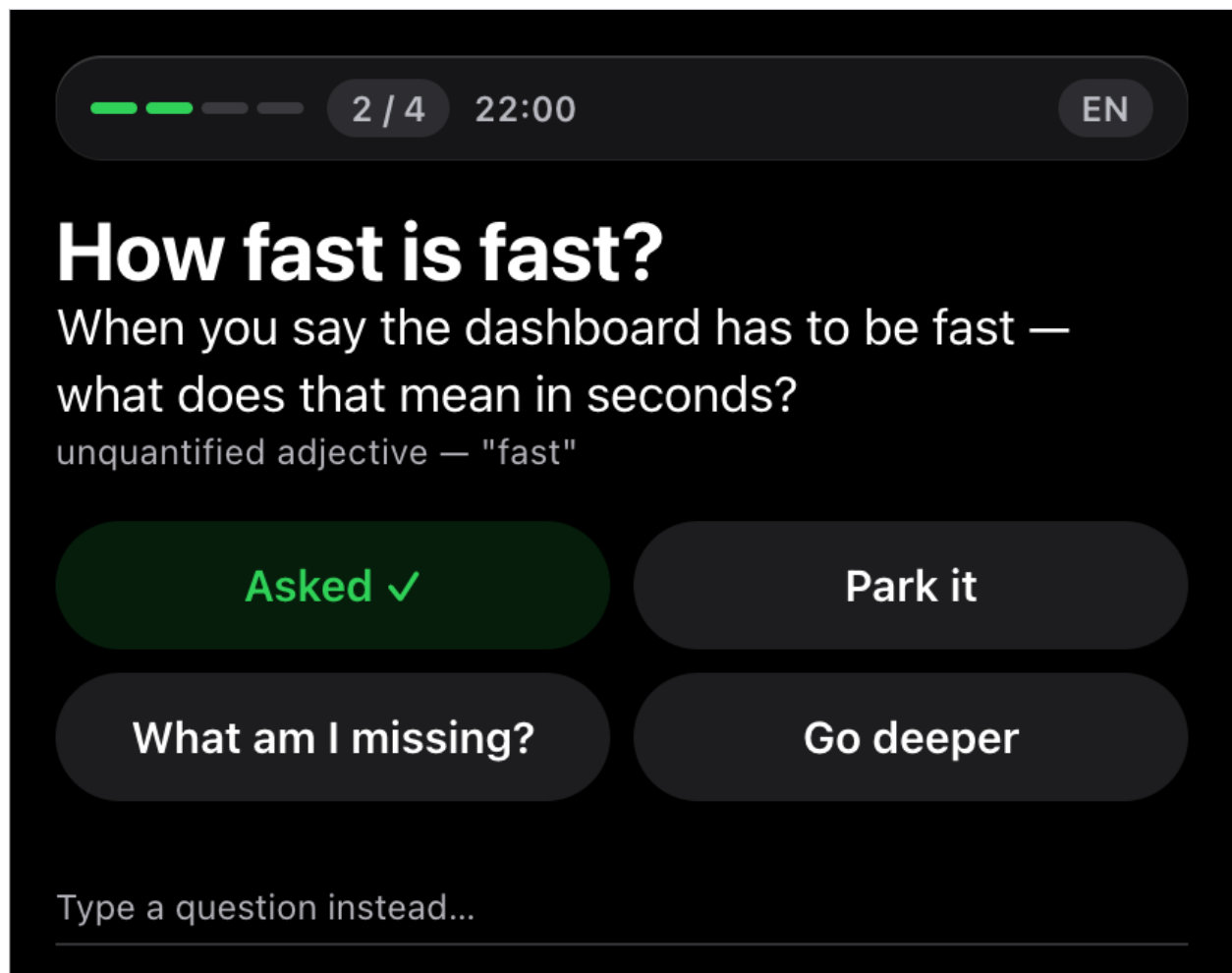
You answer in one tap

You have one hand and no attention, so every response is a single tap.

The four responses

	What you tap	What it means
You covered that topic	Asked it	The section fills in immediately
Fair question, wrong moment	Park it	It is kept for the write-up
That answer opened something	Go deeper	It follows the thread
Show me the biggest hole	What am I missing?	It names the most urgent uncovered topic

Tapping *Asked it* fills the section in front of you straight away, rather than waiting for anything to confirm. Your confirmation is what you can see.



Coverage moves the moment you tap

The way out

There is also a text box at the bottom for typing your own question. It is deliberately the quietest thing on the screen — a way out, not the way to work.

It exists because a tool that only accepts the four answers it expected will eventually be wrong in a way nobody can tell it about.

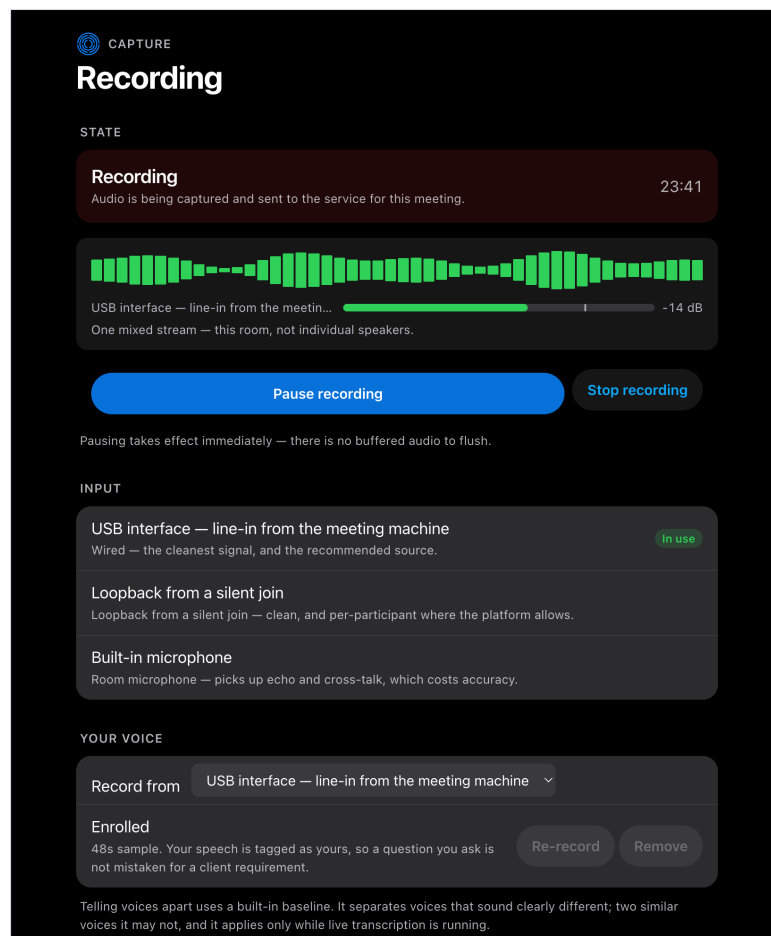
Controlling the Recording

Two of the controls on this screen are safety features rather than conveniences, and that shapes everything about how it looks.

Pause is one tap, and always in the same place

You reach for pause when a client says *"can we take this bit off the record."* A control that needs hunting for — or that asks whether you are sure — fails at exactly that moment.

So it is the largest control on the screen, never moves, and takes effect immediately. There is no buffered audio still on its way. Stopping sits beside it, deliberately smaller: stopping ends the meeting's recording and hands the microphone back, and it is not the thing you want under your thumb when you meant to pause.

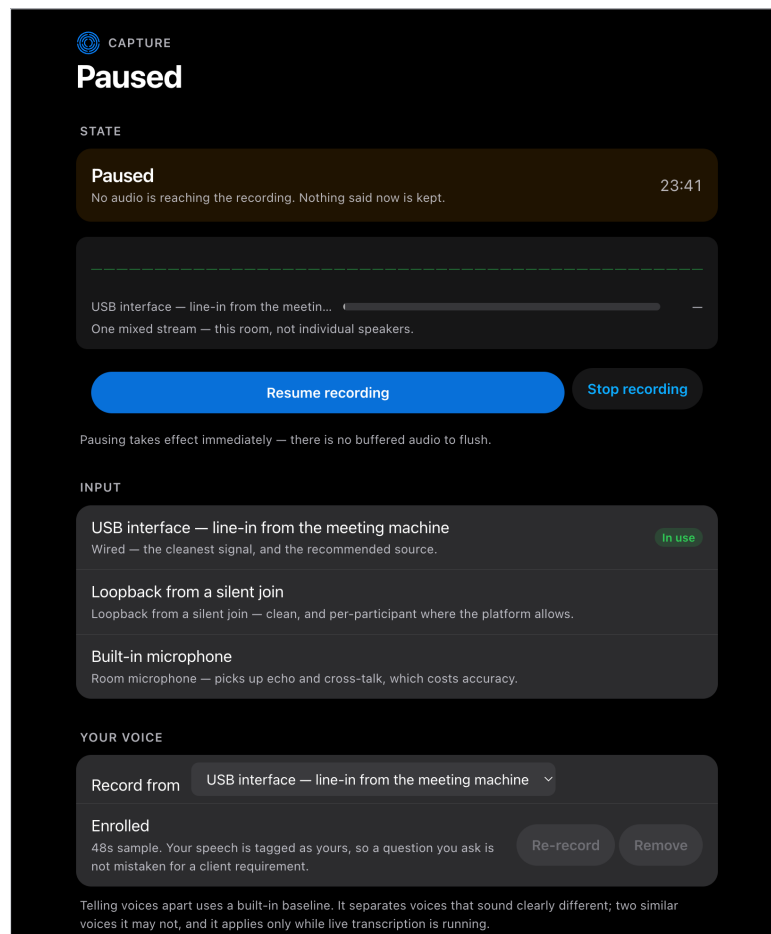


Recording, with pause one tap away

You can tell the state without reading carefully

Believing you are paused while still recording is the worst thing this product could do to you.

So the state is said three ways at once — the word, the sentence under it, and the colour — and never by colour alone, which also means it works for anyone who cannot distinguish those colours.



Paused, and saying so unmistakably

Audio captured while paused is discarded rather than held back, and resuming is instant because the microphone is never actually closed.

Starting, and choosing what to listen to

Before anything is recording, the screen offers the inputs it can see and two controls: one to check a microphone, and one to start recording. Where there is more than one input, you pick which before you start rather than discovering afterwards that it took the wrong one.

Checking opens the input and shows you the level moving, and records nothing. It is worth doing every time. A muted or unplugged microphone produces a screen that looks exactly like a working one, and without a level to watch the failure only surfaces in the transcript, long after the meeting.

Take the choice seriously at this point, because it sticks: once a recording is running the input cannot be swapped. Changing it means stopping and starting again, and that leaves a hole in the recording.

Running Elicita in a web browser rather than as an installed application, the microphone is the browser's to give, and browsers only hand it over on a secure page. On an address that is not secure, the screen says so plainly rather than claiming the feature is missing — the difference matters, because that one is yours to fix and takes a moment.

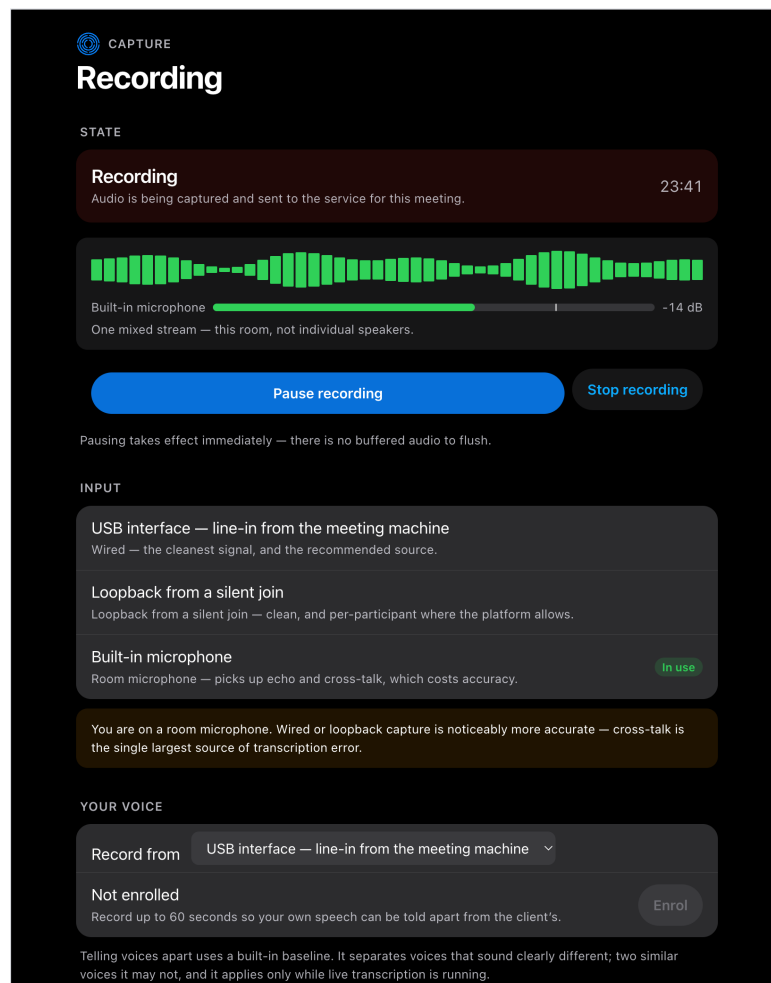
The browser also asks your permission the first time, and it asks when you press the button to check rather than the moment the screen opens. Until you have allowed it, the browser will not tell Elicita the names of your microphones, so the list shows a single unnamed input; allow it once and the real names, and any other inputs, appear from then on. That is the practical reason the check exists as its own step — it is what turns the list into a choice.

Starting the recording is also what registers the meeting. The input is opened first, so if it cannot be opened, no meeting is registered at all and you are free to fix the problem and try again. If you were already checking a microphone, that is the one that records, with no second permission prompt and no gap.

It will argue for a better microphone

Elicita does not simply accept whatever audio it is given. A wired input, or joining the call as a silent participant, is markedly more accurate than a microphone in the middle of a table. People talking over each other causes more transcription errors than the choice of transcription service does.

So a room microphone earns a warning that stays on screen, rather than a dialog at the start that everyone dismisses without reading.



On a room microphone, with the warning that earns its place

Record your own voice once

Record up to a minute of yourself, once, and Elicita can tell your speech from the client's.

It is worth the minute. Without it, a question *you* asked can be recorded as something the client wanted — and that error carries all the way into the write-up, where nobody would know to doubt it.

Pick the microphone to record it from — the list sits directly above the button, and it is a separate choice from the input the meeting is recorded on. They are different jobs. The meeting wants the cleanest feed of the room; this wants whatever is nearest your own mouth, which is often a headset the meeting is not running through at all.

Then press Enrol and talk. The screen counts the seconds as they go and shows the level beside the count,

so you can see it is hearing you rather than find out at the end that it was not. It stops itself at a minute; you can stop earlier, or discard the take and start again. What Elicita keeps is a small numerical description of your voice, not the recording — the audio is used to work that out and then thrown away. Re-recording replaces what was there before.

If you would rather Elicita held nothing of your voice at all — you are handing the laptop on, or you simply never wanted a description of your voice left on it — Remove erases it. It asks once before it does, because this is the one thing you can remove in Elicita that does not come back: nothing keeps a copy to restore from, and starting again means recording a fresh minute. Removing it takes nothing else with it. The meeting still records and still becomes a transcript; it just stops separating your speech from the client's, exactly as though you had never recorded a sample.

Where this is honest about itself

Pausing, the state display and the argument for a better microphone all work, and they work in the installed app rather than only in principle.

Recording your own voice works too, now. For a long time it did not: the screen offered it, the button did nothing, and the words above it read "Not enrolled" however many times you had pressed it.

What compares voices afterwards is a plain measurement of the shape of a voice rather than the trained model this is eventually meant to use. It separates voices that sound clearly different. Two voices that sound alike it may not separate, and when it is unsure it treats the speech as the client's — so the cost of it being wrong is a question you did not need, rather than a client's requirement quietly passed over. The screen says as much, next to where you record.

Capture no longer needs the installed application. In a browser on a secure page, the page's own microphone is used and the browser's recording indicator stays lit throughout, so the room can see the recording is running. What the installed application still has to itself is everything the browser cannot reach: the wired inputs and the silent join.

Where recording cannot start, the screen says which of the reasons it is — the page is not secure, the browser offers no microphone access, or the machine has no microphone attached — and disables the controls rather than sitting there looking ready. That last one was found the hard way: the controls went grey on a perfectly ordinary page and nothing on the screen accounted for it.

Running a Meeting in Two Languages

A bilingual meeting is not an English meeting with a few foreign words in it. Requirements arrive in both languages, and the vague answer you need to chase is as likely to be in one as the other.

Both languages, visibly

The panel shows which languages are in play, and how well it supports each one.

Two different things put a language there, and the panel says which. One it is **listening for**, worked out from what you told it about the client — a Shenzhen depot team implies Mandarin before anybody speaks. One it has **heard** is a language the transcription actually found. An expectation is drawn as an expectation; only a language actually heard carries a support badge, because support is measured from what was said and there is nothing to measure until something is.

The screenshot shows a dark-themed interface. At the top, there's a status bar with a progress indicator (a green bar followed by three grey bars), a tab indicator '1 / 4', a time '22:00', and language buttons 'EN' and 'ZH'. A 'TIER 1' badge is highlighted in green. Below this, the main text reads 'How many, exactly?' in large white font, followed by 'You mentioned 三百五十万 orders — is that 3,500,000 a year or a month?' and a subtitle 'unquantified quantity — numeral grouping'. A horizontal line separates this from the next section, which asks 'How fast is fast?' with the subtitle 'unquantified adjective — "fast"'. Below this are four rounded buttons: 'Asked it' (blue), 'Park it' (dark grey), 'What am I missing?' (dark grey), and 'Go deeper' (dark grey). At the bottom, there's a text input field with the placeholder 'Type a question instead...'.

A suggestion in a meeting running in two languages

That is not decoration. Support varies by language, and you should be able to see at a glance whether a quiet

panel means "nothing worth asking" or "I am on unfamiliar ground here."

The suggestion should be in the room's language; everything else in yours

This is the intent rather than today's behaviour, and it is worth knowing which.

The question you might read aloud should appear in the language the meeting is being held in. The short version, the reason it fired, and every label around it should stay in yours, set separately. The person reading the panel and the person being asked the question are not the same person, so they should not get the same language — a panel that assumes otherwise makes one of them work in a second language while concentrating on a client.

Today there is no setting for the language you read, and a suggestion carries no language of its own, so both are whatever the screen is written in.

Numbers

A spoken figure has to reach you as a figure you can act on: three hundred and fifty myriad has to arrive as **3,500,000**, not as words.

No transcription service handles that reliably, so Elicita converts numbers itself rather than trusting one. That conversion is written and tested in each language — and not yet connected to anything, because the transcript it would convert does not arrive yet.

This sounds like a detail and is not. A question about volume whose answer is captured at the wrong order of magnitude is worse than never asking — you would walk away believing you had a number.

Where this is honest about itself

What works today is the panel saying which languages are in play, and telling apart the ones it is listening for from the ones it has heard.

What does not is everything that depends on hearing them. Elicita has no connection to a transcription service yet, so nothing is transcribed, so no language is ever *heard*, no number is ever converted, and no phrasing is ever matched. All of that is built and waiting on one missing piece; none of it is reaching you.

The list of phrasings that carry evasion in Mandarin has been built — including the deferrals that sound like agreement and are not — but a bilingual business analyst still needs to check it against real client recordings before anyone should lean on it. Building the list was the long part; checking it needs a native speaker who does this work.

One thing worth knowing: the two languages deliberately do not cover the same ground. Mandarin catches a category of deferred commitment that English carries in tone rather than in words, which no word list could find. Each language is stronger than the other somewhere, and those gaps are named rather than assumed away.

When the Connection Drops

Networks fail in client offices. Elicita is built so that when the part of it that needs the internet cannot be reached, the meeting does not stop — it just gets quieter.

What keeps working

The fastest and most valuable suggestions never needed the internet at all.

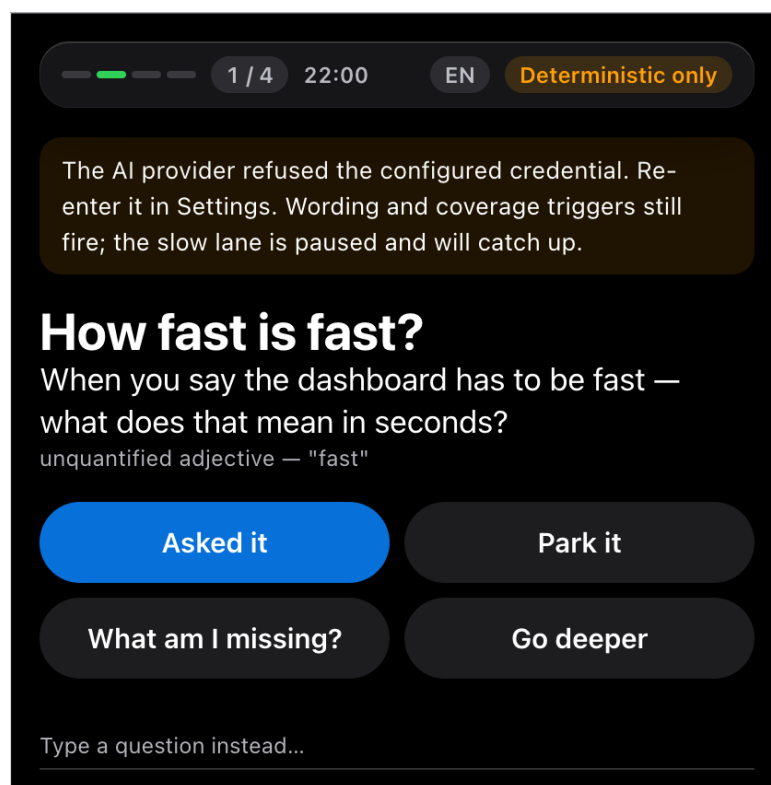
Spotting an unquantified word like *fast* is a matter of matching against a list prepared in advance, so losing the connection does not touch it. Coverage tracking and every button on the panel are local in the same way.

What pauses is the slower, cleverer layer: noticing that an answer contradicts something said twenty minutes ago, or that a system just mentioned appears nowhere in the briefing pack.

What you lose, and what you keep

	Still working	Paused
Vague words like "fast" or "a few"	Yes	—
Topics you have not covered yet	Yes	—
All four one-tap responses	Yes	—
Contradicting something said earlier	—	Paused
Spotting a system nobody briefed you on	—	Paused

You are told, not left to guess



The panel says plainly that it is running on the fast path only

The panel says which mode it is in and what that means, and it is told before it shows you a single suggestion. The alternative — going quiet and letting you assume there was nothing worth asking — is the one behaviour that would genuinely mislead you.

What it says is read from what actually happened when Elicita last spoke to the service it depends on, rather than from whether that service was set up. Those are different questions, and only the first one changes during a meeting.

It also names which problem it met, because they do not have the same fix:

Four ways the clever half goes quiet

The service cannot be reached	Usually the network in this room. Nothing to change; it comes back when the connection does.
The service is throttling you	Nothing is wrong and nothing is misconfigured. It clears on its own.
Your key was refused	Re-enter it in Settings. Waiting will not help.
Your plan does not cover it	The plan or the permission is the thing to change, not the key.

A service nobody has needed yet is reported as fine rather than as broken — most meetings never call on the slower half, and a warning shown on all of them is a warning you stop reading. When it starts working again the panel says so without anything being restarted.

The write-up says what it could not do

Nothing is invented while the connection is down. The write-up is produced in stages, and a stage that cannot reach the service stops there rather than guessing at the rest.

That leaves you with a shorter write-up, which on its own looks exactly like a meeting where little was decided. So the debrief screen tells you which stage stopped and what it reported, and you can read the rest knowing what is missing and why.

AFTER THE MEETING

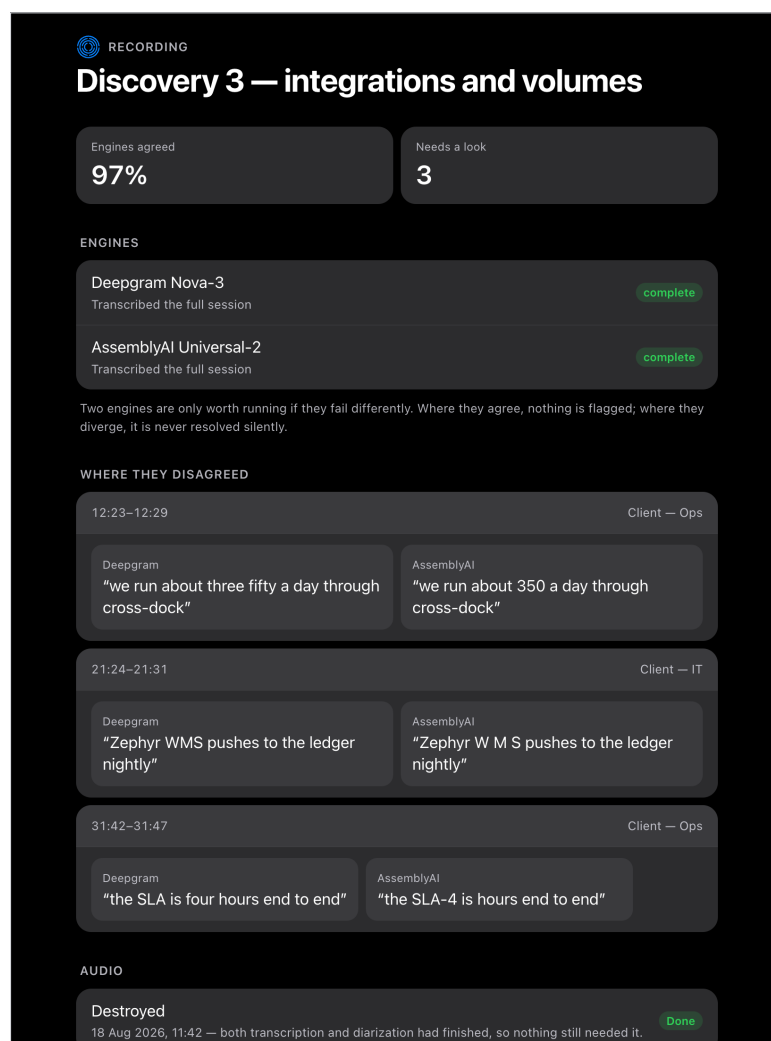
Checking the Recording

During the meeting, turning speech into text is tuned for speed — a suggestion that arrives late is worthless. Afterwards it is tuned for being right, because everything in the write-up comes from this version and not from the live one.

Two transcriptions, on purpose

The recording is transcribed twice, by two different companies.

Not for redundancy — for disagreement. Where two independent services produce the same text, it is almost certainly right and needs nobody's attention. Where they differ, something was genuinely unclear, and that is exactly what a person should look at.



Where the two transcriptions disagreed

The two readings sit side by side because you are comparing them. Stacked, it becomes a memory exercise.

Disagreements are always flagged and never quietly resolved by picking a winner. The whole value is in surfacing the doubt.

Always means any difference at all, down to a single word — and that is the case worth being explicit about. A service that hears *fifty* where the client said *fifteen* agrees with the other one about every other word in the

sentence, so a comparison that scored the sentence as a whole would call that agreement and say nothing. It is the numbers and the names that decide what gets built, and they are exactly what two services disagree about, so those are the ones put in front of you.

What does not count as a difference is where each service chose to break the text up. They do not agree about that and never will: one starts a new line wherever the room goes quiet, the other at the end of a sentence. Comparing them used to treat that as disagreement, which made two transcriptions that matched word for word come back as though they agreed about almost nothing — a paragraph shown against four words, and every line flagged. What is compared now is what was said over the same stretch of the recording, so what is left flagged is what the two services actually heard differently.

Before there is anything to check

Most of the time you will open this page for a meeting that has not been recorded yet, and that is the version of it you will see first.

The screenshot shows a dark-themed interface for 'Discovery 3 — integrations and volumes'. At the top left, there is a 'RECORDING' status with a circular icon. The main heading is 'Discovery 3 — integrations and volumes'. Below this, a large grey box contains a blue circular icon with a white dot in the center, followed by the text 'Not recorded yet'. Below this text is a paragraph: 'This meeting has not been captured, so there is no pair of transcripts to set against each other. Nothing has gone wrong — every meeting sits here until it happens.' At the bottom of this box is a blue button labeled 'Go to Capture'. Below the grey box, the text 'WHAT WILL APPEAR HERE' is displayed. Underneath, there are three sections: 'Two transcripts, made independently' with the text 'Two engines transcribe the same audio without seeing each other's work, so the mistakes they make are different mistakes.'; 'Only where they disagreed' with the text 'Agreement needs no review. What lands here is the short list of moments the two readings part company.'; and 'Then the audio is destroyed' with the text 'As soon as transcription and diarization have both finished, the recording itself is deleted and this screen says when.'

A meeting that has not been recorded yet

It used to answer that by drawing the full page with the values taken out — two figures reading "—" and "0", an empty box where the two services would be named, and nothing to do next. That is the same picture the

page would draw after a comparison that ran and found nothing wrong, and those two situations could not be further apart: one is work still to come, the other is a result.

It now says which one it is, says plainly that nothing has gone wrong, and offers the one step that would fill it. What is listed underneath is what the page will hold once a recording exists — worth saying while there is room to say it.

When one of them could not do it

Each service is listed by name with what became of it, and if one could not transcribe, it says why in that service's own words — no credential, nothing to transcribe, a refusal from the company. That sentence is the one thing worth reading on this page when a figure looks wrong, and for a while it was the one thing the page would not tell you: it printed a fixed line instead, which said the other service had produced a transcript whether or not it had. When both had failed it said that twice, about a meeting with no transcript at all.

It also shows the most recent attempt rather than every attempt ever made. Retrying a meeting a few times used to leave every one of those tries listed, so one meeting tried nine times read as nine broken services.

It is still there tomorrow

A checked recording is not something you have to deal with in one sitting. Closing Elicita and coming back to it later leaves the two transcriptions, the list of disagreements and the record of the audio being destroyed exactly where they were.

That is worth stating because it was not always true. Those three were held only for as long as the program was running, on the reasoning that they could be worked out again from the recording — which they could not, because the recording is deleted as soon as the transcribing finishes. Reopening the app showed a meeting you had certainly recorded as one that had never been recorded at all.

Then the audio is destroyed

The moment both transcriptions and the speaker identification have finished — whether they succeeded or failed — the recording is deleted, and the deletion is recorded.

Nothing still needs it, and keeping it only widens what could be exposed if something went wrong later.

Where this is honest about itself

The comparing, the flagging and the destroying are built, and the transcribing is now connected: both companies are reached with the credentials you put in Settings, and the service says at startup which of the two it can actually speak to.

Configure neither and the honest behaviour is unchanged — the job is accepted, the two engines are named, and no disagreements are reported, because nothing was transcribed. A page reporting perfect agreement over nothing is the one result that would be worth distrusting, so read the setup screen before you read the figures.

The screen says exactly that rather than putting a figure on it. It used to report that the engines had agreed on none of the recording, directly beside a count of nought disagreements — two numbers contradicting each other in the same row. Neither was a measurement. Where nothing has been compared it now says so, and that reads differently from two engines that were compared and matched throughout.

The count beside it took longer to correct than the percentage did. A nought in that place is a claim about a review that happened and turned nothing up, so where there is nothing to count no count is offered.

There is also a way to measure how much the pairing catches, once it runs: given a recording and a known-correct transcript, it reports what share of the mistakes the two services would have shown you. A sample containing no mistakes reports "cannot tell" rather than a clean pass, because it is no evidence either way.

That measurement has not been run against real client recordings. The answer can be produced; it has not been produced.

Getting the Write-Up

The meeting produces a transcript. What you actually need is a set of documents somebody can act on, where every claim can be traced back to the moment it came from.

What comes out

The documents

- 1 The open questions**
Ranked by how much they matter to the build, so the list is a plan rather than an inventory.
- 2 A log of decisions**
What was settled, and by whom. This is the part people forget and then argue about.
- 3 A draft project brief**
What is being built, for whom, and why — assembled from what was said rather than from a template.
- 4 A follow-up email**
Covering what you heard and what you still need, for you to read and send. It is never sent for you.


DEBRIEF

Discovery 3 — integrations and volumes

Every claim below links to the moment it came from. Nothing is written without one.

PROJECT BRIEF

Northwind needs Zephyr WMS integrated with the finance ledger and the carrier portal, with same-day visibility of consignment status for operations staff.

"Zephyr WMS pushes to the ledger nightly, and ops can never see where a consignment actually is." — Client — IT, 21:24

inferred

OPEN QUESTIONS

What does "fast" mean for the operations dashboard, in seconds?

"The dashboard just has to be fast, that is the main thing." — Client — Ops, 08:15

stated

Is nightly ledger sync acceptable at go-live, or is real-time required?

"Zephyr WMS pushes to the ledger nightly." — Client — IT, 21:24

inferred

Does SLA-4 mean four hours end to end, or four hours per hop?

"The SLA is four hours end to end." — Client — Ops, 31:42

stated

DECISIONS

Carrier portal integration is in scope for phase one.

"Carrier portal has to be in the first release, that is not negotiable." — Client — Programme, 38:07

stated

Reporting is deferred to phase two.

"Reporting can wait — we can live with exports for a while." — Client — Programme, 39:50

stated

The write-up, with every claim showing where it came from

Said, or worked out

This is the distinction a reviewer cares about most, so it is on the face of every claim rather than buried in it.

Two kinds of claim, always labelled

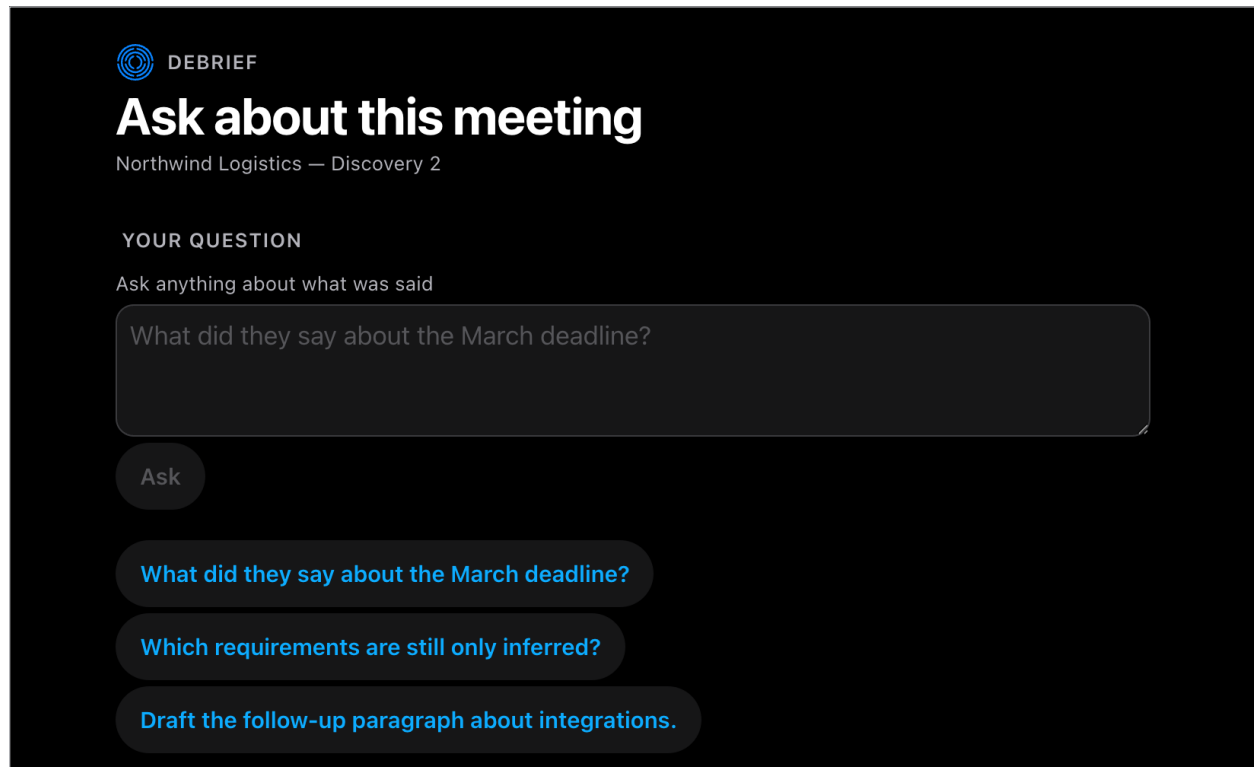
	Stated	Inferred
Where it came from	The client said it	Elicta concluded it
What is attached	The quote, with who said it and when	The quote it was concluded from
What you do with it	Check the wording	Judge whether you agree

Blurring those two would make the whole document untrustworthy, so nothing can be written without a source attached. It is not a habit or a review step: a claim without a source cannot be saved at all.

Asking about the meeting in your own words

The documents answer the questions somebody anticipated. For the ones nobody did — *what did they actually say about the March deadline, draft me the paragraph about integrations* — you can just ask.

Before you have asked anything, the screen offers a few openers and says plainly where its answers come from.



The conversation before anything has been asked


DEBRIEF

Ask about this meeting

Northwind Logistics — Discovery 2

CONVERSATION

YOU

What did they actually say about the March deadline?

ELICTA

Priya Raman (Ops, 08:15) said “the pilot has to be live before the March peak — after that nobody has time to look at it.” She did not give a date, so March 31 is inferred from “before the March peak”, not stated. It is still an open question.

YOU

Draft the follow-up paragraph about integrations.

ELICTA

Suggested: “As discussed, the pilot covers the dispatch and telematics feeds only. The warehouse management system is out of scope for this phase — we agreed to revisit it once the dispatch integration is stable.” This rests on Dan Okoro (IT, 22:40) and the scope note in your engagement brief.

YOUR QUESTION

Ask anything about what was said

What did they say about the March deadline?

Ask

Asking a question about the meeting in your own words

Answers quote and attribute rather than summarise, and say plainly when something was worked out rather than said. That is the same rule the documents follow, for the same reason: an answer you cannot check is one you have to take on trust, and this is not a product that asks for trust.

If something goes wrong partway

The write-up runs as a sequence of steps. If one of them fails, the sequence stops there and says so, rather than passing invented material to the next step.

You get a partial result and an explanation, which is recoverable. You do not get a complete-looking document with a fabricated section in it, which is not.

Where this is honest about itself

Asking follow-up questions in your own words works, against the real service, and was watched doing it.

The documents are a different matter. The write-up is built in stages, each one feeding the next, and a stage that cannot do its job stops there rather than guessing at the rest. The first stage is telling the voices apart, and that needs a transcription service Elicita is not yet connected to — so the sequence halts at the first step and none of the documents is produced. Everything after it is finished and waiting on that one piece, which is the same piece missing from the live suggestions and the recording review.

When a run stops, the screen tells you which step it stopped at and whether something was not set up or a call did not get through, so a short write-up is never left looking like a meeting where little was decided. That sentence is written for you; the step's own record, which is written for whoever maintains the software, is deliberately kept off the screen.

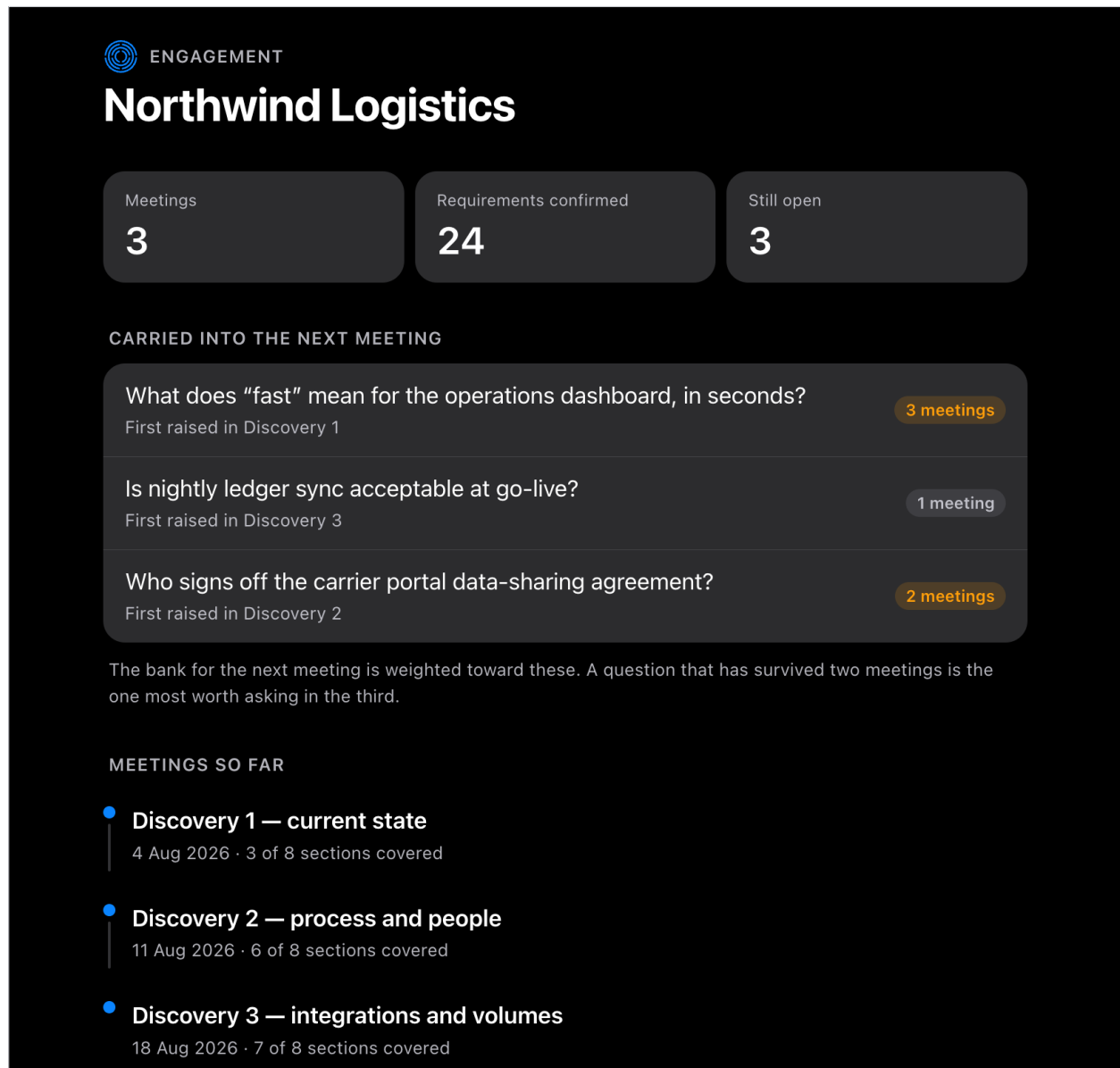
Before any run has happened at all, it says that too. A meeting whose write-up has not been made yet used to show two empty headings and nothing under them, which reads exactly like a meeting where nothing was agreed — and those are worth telling apart. The screen now says which of the two you are looking at, and that the write-up follows the recording on its own rather than waiting to be asked for.

Carrying It Into the Next Meeting

Requirements are not settled in one sitting. Everything that makes a tool which follows a whole engagement better than one that forgets after each meeting lives here.

What survives

The confirmed requirements, the decisions, and — most usefully — the questions that are still open.



What carries into the next meeting

Open questions lead the screen, ahead of completed work. That ordering is the point: what you settled is behind you, and what you did not is what you are walking back in to deal with.

The question that keeps surviving

A question that has come through three meetings unanswered is flagged.

That is worth knowing before you walk in, because there are only two explanations and both matter. Either it

is genuinely hard, or it is being deflected. Flagging it puts the choice of how to handle that in front of you while you can still plan for it.

The intent is that the question list for the next meeting leans towards these, so you do not start from scratch and are not re-asked what was already settled. That part is not connected yet — see the end of this chapter.

What is deliberately not kept

Only those things above are stored. The working notes produced along the way during a write-up are rebuilt from the recording rather than kept.

That is slower. It is also correct: a stored copy could go stale against the recording it came from, and a stale copy is worse than none, because nothing about it looks wrong.

Where this is honest about itself

Keeping the open questions, the decisions and the confirmed requirements works, and the screen above reads them.

The head start does not. Open a second meeting in the same engagement today and its question list starts empty rather than leaning on what is still unanswered: nothing yet rebuilds a meeting's questions from the engagement's carried state.

So the part you read before walking in — what is still open, and how long it has been open — is there. The part that would save you the preparation is not.

RUNNING IT WELL

Judging Whether the Suggestions Are Any Good

Whether a suggestion was worth interrupting a meeting for is a judgement, not a measurement. So it is made by a person, on real meetings — and it decides whether a new version is allowed to ship.

This is for whoever is responsible for the tool across engagements, rather than for the person running a meeting.

Replay a real meeting

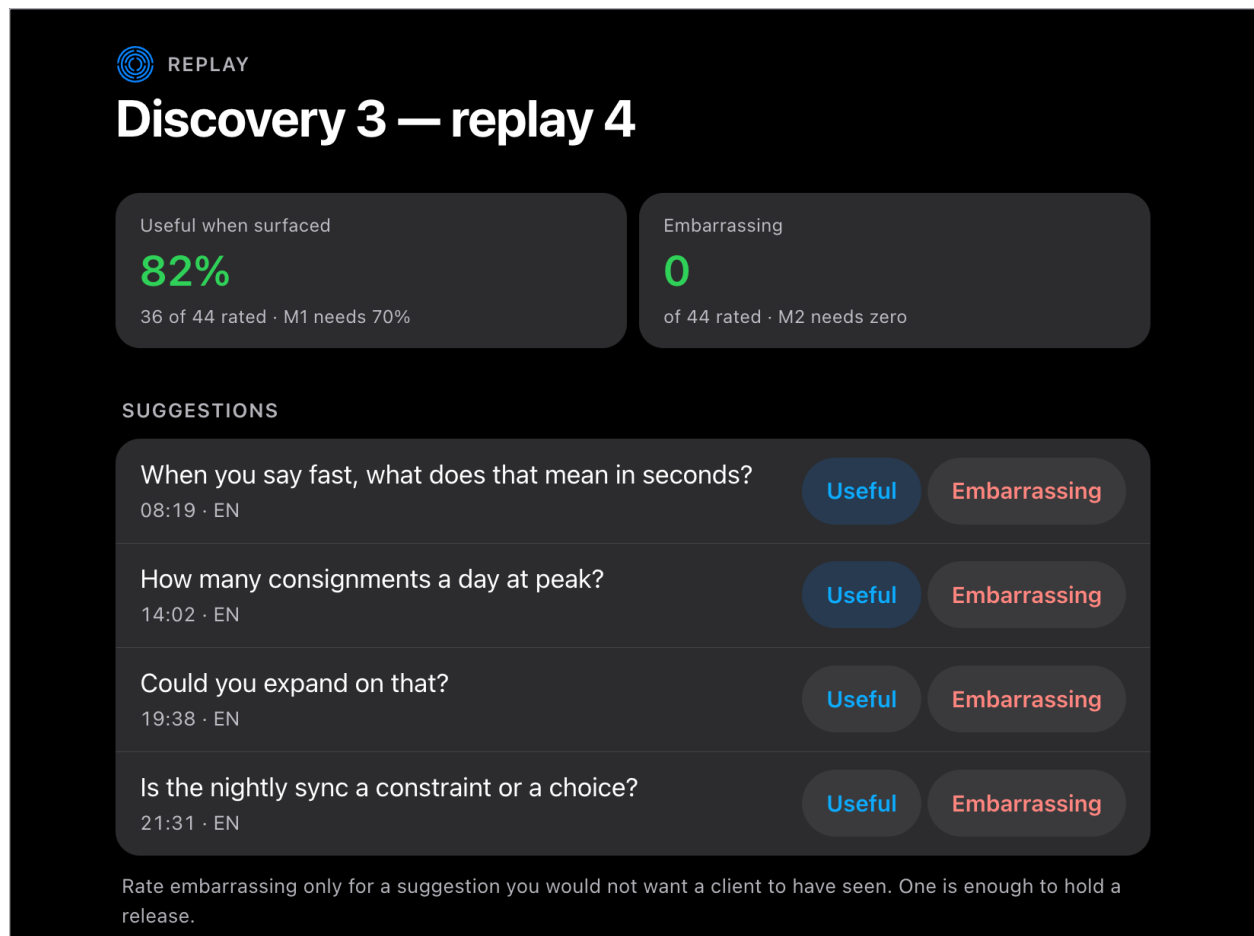
A recorded meeting is run through Elicita again, exactly as it happened. Nothing is sent anywhere and nothing costs anything: it is replaying its own past behaviour, which also means the result is repeatable.

You then rate what it suggested, on two separate scales.

Two ratings, deliberately not opposites

	Useful	Embarrassing
The question it answers	Was this worth the interruption?	Would I have minded the client seeing it?
A suggestion can be	Useless without being humiliating	Humiliating regardless of how apt it was
What a bad score costs	A wasted moment	Trust you do not get back

They are kept apart on purpose. Combining them into a single score would let a high usefulness rating average an embarrassment away, and an embarrassment is not the kind of thing that should be averaged.

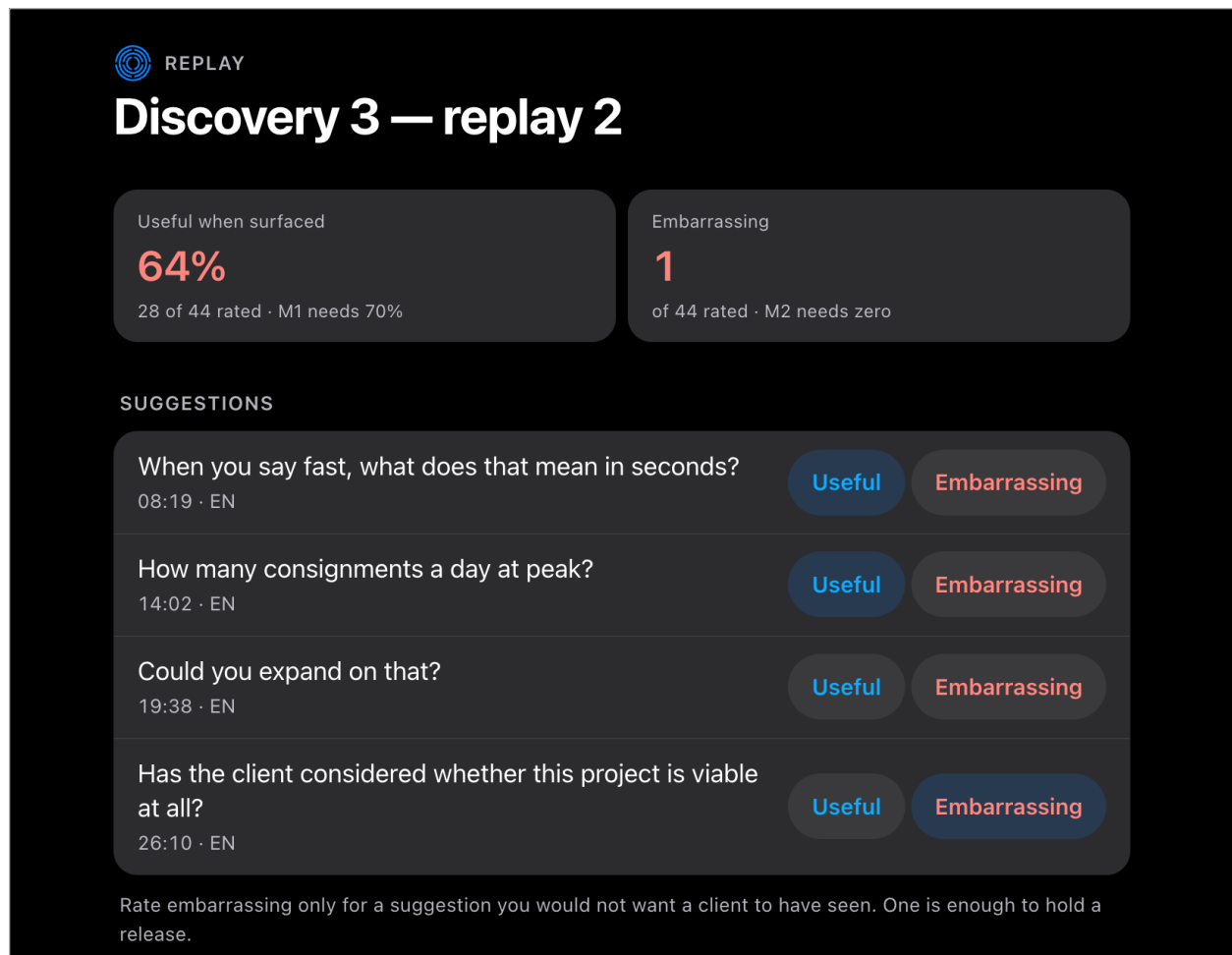


A version that clears both bars

Two bars, and one of them is absolute

At least **70 per cent** of suggestions must be rated useful.

The number rated embarrassing must be **zero**. Not low — zero. One is enough to hold a release.



A version that fails both

A version that fails either bar is blocked automatically. Nobody has to remember to check.

Where this is honest about itself

Replaying, recording ratings, both bars and the automatic block all work today.

Each bar now shows what it was measured over, which matters more than it sounds. A figure of "useful nine times in ten" means one thing across four ratings and quite another across four hundred, and the screen used to show only the percentage — once reporting a comfortable pass, in green, from a single rating. A run with nothing rated yet says so, rather than claiming either a pass or a failure it has no grounds for.

What is missing is the way in. A run's individual suggestions are not yet listed on the screen, so there is nowhere to sit and rate them; the ratings behind these figures were recorded another way.

Both bars are judged separately for each language, so adding a language means finding a fluent rater who also does business analysis — not just translating a word list. That is worth planning for early, because it is a hiring problem rather than a software one.

What It Will Not Do

It is easier to trust a tool when you know where its edges are. This is the list of things Elicita deliberately does not do, and why each line was drawn.

It never speaks to the client

It has no voice in the meeting. It does not send messages, and it does not act on anybody's behalf. Every suggestion is shown to you, and you decide.

If you ignore it for ninety minutes, nothing happens. That is a supported way to use it, not a failure.

It is not a transcription product

It does turn speech into text, because it cannot work otherwise. But that part is bought in, and it is not what Elicita is for. If all you want is a recording and a transcript, this is a great deal of machinery for something simpler tools already do well.

It is not a note-taker

Notes come out of it, as a by-product. The purpose is the question asked in the room. A tool that captured the meeting perfectly and never improved what was asked would have missed the point entirely.

It is not something you talk to during a meeting

You cannot ask it things while the meeting is running. There is no conversation with it. It offers; you accept or decline.

That restriction is deliberate. Anything that invites you to start typing has taken your attention away from the client, which is the exact problem it was built to relieve. Asking it things in your own words is something you do afterwards, on a different screen.

It stays quiet when it is unsure

Given the choice between a suggestion that might be useful and saying nothing, it says nothing.

One suggestion that embarrasses you in front of a client does more damage than ten good ones repair. So the bar for speaking is set high, and it is checked against recordings of real meetings before every release.

The practical effect is that Elicita will sometimes miss something a sharp analyst would have caught. That trade was made knowingly, and it is the right way round.

It does not replace judgement

It does not decide what matters, what is worth pushing on, or when to let something go because the room has had enough.

It notices, it suggests, and it writes down what it heard. Everything that requires knowing this client, this project and this moment stays with you.

What Works Today

This page is assembled automatically every time this guide is made, from notes kept alongside the product itself, so it cannot fall out of date. Editing it by hand has no effect — the next build puts it back.

85 notes in total, 60 of them describing something that works today.

This page is read from the notes kept alongside each part of the product, so it says what is true now rather than what was true when somebody last remembered to update a page.

Working today

You can rely on these.

Part of the product	What the note says
Prepare for the engagement	Creating engagements and the meetings inside them — the first one and every one after it, and naming or removing any of them afterwards; adding documents by dropping the files in or by link, and tagging them; reading what is inside them; the client vocabulary list; and reviewing, reordering and pruning the question bank — all of it from the screen, and all of it within one sitting
Prepare for the engagement	All of it is kept. The engagement, its meetings and what you named them, the documents you attach and their contents, the words you add, and the question bank are written to a single file on your machine as you go. Nothing needs installing for that, and a firm that wants everything on its own database server can point Elicita at one instead
Prepare for the engagement	Taking things back out. A document, a word, a single meeting or the whole engagement can be removed from the screen. Nothing is erased: the record is kept and can be brought back, which is why removing a client or a meeting here is a tidying-up action and not a way to honour a request to be forgotten
Prepare for the engagement	Collecting the drafted questions. Drafting is submitted as a job that finishes minutes later; the service now goes back for the result every thirty seconds, puts the questions in the bank, and stops asking once a job has ended, failed or expired. A compile that stops for any other reason now says so in the log instead of leaving an empty bank and no explanation
Prepare for the engagement	The screen says why the bank is empty. Compiling answers "accepted" whatever happens next, and the bank answers an empty list whatever the reason, so between them they said nothing at all about four consecutive failures — the screen read "Not compiled yet" every time, after four compiles had run. It now names the step that stopped and what to do: nothing set up, a credential that may not use the model, a token that cannot submit the drafting job, a provider being throttled, or a network that could not be reached. Those have four different fixes and one of them is not "wait"
Prepare for the engagement	A refusal that waiting cannot fix is no longer filed as a throttle. A model outside a credential's plan is refused with the same status as a rate limit, and Elicita read it as one — its own message said "check the model in Settings" while the category it was filed under said "try again shortly". Anything reading the category, which is what a screen must do, repeated the advice the message had just ruled out
Prepare for the engagement	Compiling no longer holds the request open. Compile answered "accepted" only once the whole chain had run — measured against a real provider, 45 seconds for a status whose entire meaning is that the work is happening elsewhere. Any client giving up at thirty seconds saw a failure for a compile that was running perfectly well, and pressing the button again started a second one. It now answers in milliseconds, the screen says while it is working, and the button will not start another on top of it

Part of the product	What the note says
Prepare for the engagement	A credential that cannot submit a batch job can still draft a bank. The drafting pass is handed to a cheaper, slower way of asking that some accounts are not permitted to use, and that was the only way it was ever asked — so those accounts could never produce a bank at all, however many times Compile was pressed. Where the cheap route is refused for want of permission, the same pass is now asked the ordinary way instead. Only a permission refusal falls back: a provider that is simply unreachable will answer the second call exactly as it answered the first
Prepare for the engagement	A bank is drafted end to end, on this credential. The whole chain runs against a live model — reading the documents, sorting what is in them, and drafting the questions — and the questions reach the screen. Two things had to change for that. The acceptance check demanded at least 150 questions, which is what a good pass over a real document set produces and not what an acceptance check should insist on: a perfectly good pass over three short documents came back with twelve and was thrown away whole. That check now catches a pass that did not really answer, and leaves judging the volume to whoever reads the bank
Prepare for the engagement	The bank is filed under sections somebody chose, not sections a compile invented. Nothing ever told the drafting pass which sections exist — it was asked to use "the sections the documents imply", so it invented one called <i>Operations</i> and put 65 of 97 questions in it. That is a bin rather than a section: it cannot be reviewed section by section, two engagements filed under different sections cannot be compared, and coverage is tracked against sections the meter has to have heard of. The sections now travel with each request as a closed list, and a live compile spread 73 questions across all eight, the largest section holding eleven
Prepare for the engagement	The drafting pass is asked for the right thing. It was being sent the <i>debrief's</i> instructions — produce open questions, a decision log, a project brief and a follow-up email, citing the transcript. There is no transcript before a meeting, and what is wanted is a bank of questions tagged with the template section each one serves. The model did as it was told: a live compile filed its candidates under "open questions" and "decision log", and half of them were statements rather than questions. The two passes now have their own instructions
Start the meeting	The whole audio-handling policy, including automatic destruction, and — wherever an engagement asks for consent — the gate that holds recording shut and the record of who confirmed it
Start the meeting	This screen is now only the gate. It books nothing and opens nothing: it confirms consent, states what happens to the audio, and offers one way on to the recording, which is unavailable until consent is settled. It used to carry a Start button that registered a meeting and opened no microphone, above a row showing the meeting's <i>language</i> mode under a heading saying Capture
Start the meeting	Choosing an input, checking it, and recording on it all happen on the recording screen, which is the one that can show a level meter. It works in the desktop app — which offers the audio interface and the silent-join capture path — and in a browser on a secure page, it warns you off the room mic, and the recording survives moving between screens
Start the meeting	Elicita will join meetings as a silent participant through a single meeting-bot service that covers every platform, rather than integrating with Zoom, Teams and Meet separately. That keeps one integration instead of three, and gives us who-said-what as a fact the platform reports rather than something guessed from the audio. The connection point is built and tested; the vendor contract is a commercial step, not an engineering one
Catch a vague answer while it still matters	The panel itself: the running coverage count and time remaining, the suggestion and the line saying why it fired, all four one-tap responses, and the box for typing your own. It is connected to the meeting's live session, so whatever that session sends appears here
Catch a vague answer while it still matters	Your taps are recorded against the meeting and sent back, so the debrief knows which suggestions you used and which you set aside

Part of the product	What the note says
Catch a vague answer while it still matters	The microphone. The desktop app opens the device, offers the audio interface and the silent-join path, and releases it when you stop
Catch a vague answer while it still matters	Something to put in the panel. What the microphone hears is turned into words during the meeting, checked for the kind of vagueness that costs a requirement — an amount with no number, an adjective with no threshold, a date that is not a date, a commitment behind a hedge — and a question is put in front of you. Verified end to end against a real recording
Catch a vague answer while it still matters	The question comes from the bank drafted before the meeting, chosen for the word you actually heard rather than for how highly it was ranked. Where the bank has nothing about that word, the question is built from the word itself, so the panel still works when no model can be reached
Catch a vague answer while it still matters	At most one suggestion a minute, however much passes the check. The limit matters more than it sounds: a panel that interrupts more often than that stops being read, and it stops being read for the rest of the meeting
Run a meeting in two languages	Showing which languages are in play, and telling apart the ones Elicita is listening for from the ones it has heard. What it listens for is worked out from the client background when the engagement is made, kept with the engagement, and shown on the panel before a word is spoken
Run a meeting in two languages	We measured it rather than leaving it open. The two languages do not cover the same ground, and that is correct: Mandarin catches a whole category of deferred commitment that English carries in tone rather than words, so no word list could find it. Each language is stronger than the other somewhere, and the gaps are now named rather than assumed away
When the connection drops	The two-speed design, the panel's degraded state, and honest recording of what could not be completed
When the connection drops	The panel is told which mode it is in by the live connection itself, before it shows you a single suggestion — so a quiet panel is never ambiguous between “nothing to say” and “the model is unreachable”
When the connection drops	The mode reflects the connection, not the setup. It used to be decided by whether a provider had been configured — a fact settled when Elicita started, which no outage, expired key, throttle or plan limit could change. A journey about the connection dropping could not detect one dropping. Now every call to a provider reports what it found, and the panel names which of the four problems it met, because the remedies are different. Recovery clears it without a restart
When the connection drops	A short write-up says why it is short. When a stage of the write-up cannot reach a model the chain stops there rather than guessing, which is right — but the screen then showed fewer artifacts and no explanation, which reads exactly like a meeting where nothing was decided. The stage that stopped and the reason it gave were both recorded all along, into a place nothing read. The debrief screen now says which stage stopped and what it reported
When the connection drops	The one call that carried the most now leaves a record. The debrief conversation sends a whole meeting's transcript to a provider so you can ask questions about it, and it was reaching the provider directly rather than through the audited boundary — so it was the single call writing no entry in the log of what left the machine, and telling the panel nothing when it failed. It goes through the boundary now
Check the recording	Comparing two transcripts, flagging every disagreement, and the automatic destruction of the audio — the last of which a live run photographed working, with the reason recorded beside it

Part of the product	What the note says
Check the recording	Refusing to report an agreement it has not measured. A live run showed "Engines agreed 0%" next to "Needs a look 0" — two engines that had finished, no disagreements between them, and a headline saying they matched on nothing. Both numbers were on screen at once and they contradict each other. The screen now says nothing was compared, and separates that from the engines having agreed throughout, because an operator acts on those differently
Check the recording	Both transcribers are real, and both complete. Given the same raw, chunked linear16 audio the capture pipeline produces — posted chunk by chunk the way this journey describes, out of order once to confirm the gap is refused rather than silently joined — Deepgram and AssemblyAI both return COMPLETE transcripts from the real vendor. AssemblyAI needed a fix first: it was handed the identical headerless PCM Deepgram accepts, and its transcoder rejected it as an unrecognised file type, where Deepgram is told the encoding and sample rate on the request itself. Wrapping the same bytes in a plain WAV container before upload — no re-encoding, nothing about the audio changes — is what made it complete. The engagement's own vocabulary reached both vendors on the wire: Deepgram spelled a product term and a person's surname correctly; AssemblyAI, boosted with the same two terms, got the product term right and still misheard the surname — itself exactly the kind of disagreement this pairing exists to catch. Full diarization over the same audio told the two real speakers apart into two distinct tags rather than one — R7, open since the first spikes used a single narrator, is closed
Check the recording	A live SessionAlignment now exists, with real measured divergence. Comparing the two real transcripts over the same 57-second recording produced 13 aligned spans, agreement scores from 0.11 to 0.82, every span flagged divergent — including the misspelled surname above (Deepgram: "Peter DeCarlo"; AssemblyAI: "Peter DiCarlo"). Read plainly rather than as unqualified success: the two vendors split the recording into a different number of utterances (13 against 8), so a short span's comparison window often pulls in extra neighbouring words from the other engine, which alone is enough to fail an exact-word match — "13 out of 13 divergent" therefore overstates how differently the two vendors actually heard this recording, and a cleaner read of agreement-versus-disagreement on well-aligned boundaries has not been seen yet. What has been seen, for the first time, live and against two real vendors on real audio: the comparison runs, and it surfaces a genuine mishearing, not nothing
Check the recording	A meeting that was never recorded says so, rather than reviewing nothing. The screen used to render its own analysis with the data removed, which reads as a comparison that ran and found nothing — and it left the "Needs a look" count at nought, which is the half of the "Engines agreed 0%" contradiction above that was still on screen. A count of nought disagreements is a claim about a review that happened. Where nothing was compared there is now no count offered at all, and a meeting with no recording gets its own screen with the next step on it
Check the recording	The recording survives the service being restarted. The two transcripts, the alignment computed from them and the audio-destruction record were all held in process memory with no table behind them, classified as pipeline output "rebuilt from the transcript". Two of them are the transcript, and the third describes audio the privacy control has already destroyed, so nothing rebuilt any of them — a meeting that really was recorded came back from a restart answering 404 on all three reads at once, which is precisely what a meeting that never happened answers. They now have tables. Four tests fail on the old code for the reasons that matter: the transcript, the divergences and the destruction record each 404 after the restart, and a meeting nobody recorded still 404s, which is what stops the fix from turning "not recorded" into a comparison that ran
Check the recording	The check for that is now built: given a recording and a correct transcript, it reports what share of the mistakes the pairing would actually have shown you. A sample containing no mistakes reports "cannot tell" rather than a clean pass, because it is no evidence either way
Get the write-up	The whole sequence is built — cleaning up the transcript, translating while keeping the original, sorting statements into template sections, drafting the documents, and attaching every source
Get the write-up	If any step fails, the sequence stops there and says so, rather than passing invented material to the next step

Part of the product	What the note says
Get the write-up	And now it says so where you can see it. A run that stops leaves the screen with fewer documents on it, which reads exactly like a meeting where little was decided. The screen names the step it stopped at and whether something was not set up or a call did not get through. That sentence is written for you: the step's own record, which is written for whoever maintains the pipeline, is kept out of it
Get the write-up	Asking follow-up questions in your own words works, live and against a real provider. You can query the meeting, ask for a paragraph to be drafted, and every answer names the moment it rests on — proven in a live run, not just in principle
Get the write-up	Telling the voices apart is no longer the missing piece, and the sequence runs past it. A real two-speaker recording, chunked and posted through the record-path API exactly as this journey describes, produced a COMPLETE transcript from a real speech vendor; full diarization over that same audio told the two real speakers apart, and every stage after it — cleanup, translation, section classification, the analyst chain, citation binding, state merge — ran to completion on real content. Nothing in this sequence was a fixture standing in for a vendor
Get the write-up	The four documents — plus the transcript and coverage matrix — came out of a real meeting, each claim citing the real recording and naming which of the two real speakers said it. The open questions and project brief both carry citations from either speaker, correctly naming the person the recording actually named (a keyterm from the engagement's own vocabulary, not guessed). Both configured vendors completed on this run (see journey 6 for the fix that made the second one complete) — the write-up drafts from one engine's reference timeline rather than requiring both, so it would still have finished either way, but this time it did not have to fall back
Carry what you learned into the next meeting	The next meeting starts from the last one's work. A meeting's question list is built from its engagement's prepared bank, with anything still open from previous meetings put at the top — and a question you pruned stays pruned in every meeting after it. This was written, tested and connected to nothing: both halves were read from per-meeting records nothing ever wrote, so every meeting of every engagement served an empty list. A live run failed on it with an empty bank and a fresh timestamp. Both halves already existed on the engagement; they only needed joining
Carry what you learned into the next meeting	All of it now survives restarting the service. The client, its meetings, the open questions, the standing requirements and the prepared question bank are written to a database as they change, and read back when Elicita next starts
Carry what you learned into the next meeting	Only the five things above are stored. The working notes a debrief produces along the way are rebuilt from the recording rather than kept — slower, but a stored copy could go stale against the recording it came from, and a stale one is worse than none
Judge whether the suggestions are any good	Replaying meetings, recording ratings, both quality bars, and automatic blocking of a release that fails either
Judge whether the suggestions are any good	Both figures now say what they were measured over. A live run photographed "Useful when surfaced 100%", in green, against a bar of 70% — taken over a single rating, with nothing on screen to say so. These two numbers can hold a release, and neither can be read without knowing how much evidence is behind it. The release check has always printed the fraction; the screen printed the percentage alone. A run with nothing rated now reports that, rather than a pass or a failure it has no grounds for
Judge whether the suggestions are any good	The starting weights are now chosen rather than left at a placeholder, and the reasoning is written down: say less, more confidently, because a suggestion that misfires in front of a client costs more than one that never fires. A question you have already asked can never be suggested again ahead of one you have not — that is guaranteed by the numbers themselves, not left to chance

Part of the product	What the note says
Set up the services Elicita uses	Everything on this screen, including live credential testing for the AI provider and both supported transcription services
Set up the services Elicita uses	For a shared deployment the key can now come from your own secret store — 1Password, Vault, AWS, Google, anything with a command line. If the store cannot be reached, Elicita refuses to start and says why, rather than quietly making a new key that leaves the other machines unable to read anything
Control the recording while the meeting runs	Pause and resume, the state display, and the input warning
Control the recording while the meeting runs	Pause now pauses a real microphone. Audio captured while paused is discarded rather than held back, and resuming is instant because the device is never closed
Control the recording while the meeting runs	A control that cannot work now says why. A live run photographed a greyed-out "Start recording" beside an empty input list, with no explanation anywhere on the screen — the page was secure and the browser offered microphone access, so both existing checks passed and the screen concluded there was nothing to explain. The machine simply had no microphone. That case now says so, because an operator can act on "no microphone" and can do nothing at all about a dead button
Control the recording while the meeting runs	A check before the meeting, and a choice of input. Check microphone opens the chosen input, shows a live level and records nothing — which is also what makes a browser reveal the real names of its microphones, since it withholds ids and labels until the first permission grant and the picker before that reads "Microphone 1". The checked device is promoted to recording rather than reopened, so there is no second prompt and no gap. It matters that the choice is made here: capture refuses a second session on either backend, so swapping input mid-meeting means stopping and restarting
Control the recording while the meeting runs	Starting a recording is what books the meeting. The consent screen used to carry a Start button that registered a session and opened no microphone, so an operator was told a meeting had begun while nothing was listening — and it offered no way to pick an input, being a screen that cannot show one working. Both moved here. The device is opened before the meeting is registered, so an input that will not open books nothing
Control the recording while the meeting runs	Recording your own voice. The button used to do nothing, and the words above it said "Not enrolled" however many times you had enrolled — there was no route behind it to enrol against. Recording now works: up to a minute, counted out loud as it runs, capped by the service rather than by the screen, and the recording itself is turned into a voiceprint and thrown away rather than stored. During a meeting your own sentences are no longer interrogated as though a client had said them
Control the recording while the meeting runs	Removing the voice sample again. The route to erase a voiceprint had been there since enrolment was built, and no screen reached it — the only thing the panel could do with an enrolment was record over it, which leaves a description of somebody's voice on the machine either way. Remove now sits beside Re-record whenever there is something to remove. It asks once first, because this deletion is the exception to the rule the rest of the product follows: everything else marked removed is only hidden, and a voiceprint is genuinely destroyed
Get Elicita onto people's machines	Packaging for both platforms, signing and verification, the managed-deployment profile, and the automatic check that both platforms behave identically
Get Elicita onto people's machines	The About screen reports the real version and platform it is running on

Part of the product	What the note says
Get Elicita onto people's machines	Whether the build is signed, and how it was installed, are now read from the operating system rather than taken on trust. Where the answer cannot be established the screen says so, which reads differently from "unsigned" — an IT reviewer needs to tell those apart
Get Elicita onto people's machines	Elicita checks for a new version on launch and tells you what is waiting. It never installs on its own — an update that restarted the app by itself would eventually do it during a client meeting

Needs setting up first

These work, once somebody has configured the outside service they depend on.

Part of the product	What the note says
Prepare for the engagement	Two one-off steps, both in Settings and both covered in journey 10. Drafting the questions needs an AI provider. Reading a document from SharePoint or OneDrive needs your Microsoft 365 tenant registered — a dropped file needs neither, which is the way in that always works
Catch a vague answer while it still matters	Hearing the meeting needs a speech provider configured in Settings; without one the panel stays at its resting state and the recording is unaffected. Once configured it is charged by that vendor for every few seconds of audio, silence included, for as long as the recording runs

Still to come

These are described elsewhere in this guide as things that are not finished. They are listed together here so that nobody has to hunt for the caveats.

Part of the product	What the note says
Prepare for the engagement	Erasing a client's data outright, and the engagement list showing more than the first twenty
Prepare for the engagement	Sections that come from the engagement's own template. An engagement names its target requirements template as free text, and nothing turns that name into a section list — so the list above is a stated default rather than the template the client asked for. The write-up still classifies against no sections at all, which is the same gap one step further on
Prepare for the engagement	Remembering a failed compile across a restart. The explanation above lives only as long as the service does. Restart it and the screen goes back to offering a compile, which is a fair trade — a failure kept from before a restart might describe a problem you have already fixed — but it does mean pressing Compile again is how you get the current answer
Prepare for the engagement	A scanned PDF. The text of a page that is really a photograph is not recovered, so such a document contributes nothing rather than contributing nonsense
Start the meeting	Asking for consent at all. Every engagement is currently set to treat consent as settled for the engagement as a whole, so no meeting asks and no consent record is written. The gate that would ask is built and tested; nothing can switch an engagement over to it yet
Catch a vague answer while it still matters	Hearing across a pause. The recogniser is given four seconds at a time and nothing listens for the ends of sentences, so a phrase landing on the join between two of them can be missed. What is heard is accurate; what is missed is silent

Part of the product	What the note says
Catch a vague answer while it still matters	The other three ways a question can be earned — an action with nobody named as doing it, an answer that contradicts a document you supplied, a system or team nobody has mentioned before. Only the wording checks are built, and they are the ones that need no model
Run a meeting in two languages	Hearing them. Detection needs a transcription service, and no client for one exists on the live path — so today the strip shows what is expected and never fills in what was heard. Everything downstream of detection is built and waiting on that one piece
Run a meeting in two languages	The suggestion in one language and the interface in another. There is no setting for the language you read, and a suggestion carries no language of its own, so today both are whatever the screen is written in
Run a meeting in two languages	Number conversion, in the product. It is written and tested — thoroughly, per language — but it lives in a component nothing calls yet, so no number reaching you has been through it
Run a meeting in two languages	Nothing reaches the Mandarin list yet, for the same reason as the rest of this page: it is read from a transcript, and nothing is transcribing. The list itself covers the phrasings that carry evasion in Chinese business speech — including the deferrals that sound like agreement and are not, such as “we’ll look into it”, and the qualifiers that quietly withdraw most of a yes
Run a meeting in two languages	A bilingual business analyst still needs to review that list against real client recordings before we would rely on it. Building it was the long part; checking it is the part we cannot do without a native speaker who does this work
When the connection drops	Detecting a drop while nothing is being asked of the provider. The mode is read from real calls, so a connection that fails between them is not noticed until the next one. There is no background health check on purpose: a probe answers a question nobody asked and can disagree with the calls that matter
When the connection drops	The slower layer itself. The periodic slow-lane turn records that it ran and reaches no model at all, so the cleverer half being described here has nothing behind it yet. That is a missing feature rather than a degraded one, and it does not change what the panel says about the connection
Check the recording	The check still has to be run against real client recordings. We can measure the answer now; we do not have it yet
Get the write-up	This is still only the after-the-meeting half. Nothing about the live path changed here: the panel still sits at its resting state for the whole meeting, no nudge fires, and audio reaches the record path only by posting chunks over the API by hand — nothing in the desktop app drives any of this yet
Carry what you learned into the next meeting	Carrying requirements and decisions into a second meeting. Those are produced by the write-up, and the write-up now runs the whole way through on a real recording — so an engagement’s confirmed requirements and decisions are written from a real meeting rather than waiting on a transcription service that did not exist. What has not been watched is the other end: a second meeting reading them back. The join is proven against records handed to it, not yet against a set a real meeting produced. Until then, what a second meeting demonstrably inherits is the open questions
Carry what you learned into the next meeting	Watching the inheritance work end to end. A second meeting inherits its engagement’s prepared bank, and on the machine this was tested against that bank is empty — the credential available here cannot submit the drafting job at all, so nothing was ever drafted to carry forward. The join is proven against a bank supplied to it; it has not been seen carrying a bank a provider actually produced
Judge whether the suggestions are any good	Rating from the screen. A run’s individual suggestions are not listed anywhere the screen can read them, so the section that should offer them to rate is empty and the ratings behind the figures were sent directly to the service. The rating itself works; the way in does not exist yet

Part of the product	What the note says
Judge whether the suggestions are any good	Both bars are judged per language, so adding a language means finding a fluent rater with business-analysis experience — not just translating a word list. Worth planning for early
Control the recording while the meeting runs	Telling two similar voices apart. What compares voices is a baseline built from the shape of a voice, not the learned speaker model the design asks for — nothing in the product could run one, and adding it is its own piece of work. It separates voices that sound clearly different and may not separate two that sound alike. It leans towards treating speech as the client's when it is unsure, so the cost of being wrong is one question you did not need rather than a client requirement silently passed over. The screen says all of this where you enrol
Get Elicita onto people's machines	The update service needs its signing key and address set before a release goes out. Until then the screen says there is no update channel configured, rather than accepting whatever it is offered
Get Elicita onto people's machines	Listing the permissions the build holds. Nothing asks the operating system what it was granted, and the screen's permission list has always been passed in empty — a live run failed on it with no rows at all. A bare heading reads as "nothing is needed", which is the opposite of true, so the screen now says the answer cannot be read here and that capture needs the microphone. The list itself is still to come